

Domains 1–2 · Software Development and APIs

Generated from the Markdown study system. Labs with executable code remain in the `Labs/` folder. This is study material, not a Cisco exam dump and not a pass guarantee.

1.0 Software Development and Design — 15%

Software Development and Design is the foundation of CCNA Automation (200-901 CCNAAUTO v1.1). The exam is 120 minutes. This domain is weighted at **15%**, which is smaller than APIs (20%) or Cisco platforms, but it is the language the rest of the exam speaks: data formats, Python structure, testing, process models, and Git.

You are not being tested as a full-time software engineer. You are being tested as a network engineer who can read, write, and reason about the artifacts that automation produces. A RESTCONF payload is JSON or XML. An Ansible playbook is YAML. A webhook is an Observer-style callback. A failed merge is a Git conflict. If you cannot compare JSON to YAML, parse either into a Python `dict`, or explain why a function is better than a 200-line script, later domains become guesswork.

This chapter teaches Domain 1 completely, then Domain 2 (Understanding and Using APIs, 20%). Domain 2 is one of the two heaviest domains on the exam; treat it as core, not as an appendix. Official Cisco developer material lives at <https://developer.cisco.com/>. Python's JSON library is documented at <https://docs.python.org/3/library/json.html>. Git's reference is <https://git-scm.com/docs>.

Nothing in this file is a claimed Cisco exam question. Practice items are original scenarios written to match the **blueprint verbs**: Compare, Describe, Explain, Utilize, Construct, Interpret, Troubleshoot.

1.1 Compare data formats (XML, JSON, and YAML)

1. What Cisco expects me to know

Compare means differences, advantages, disadvantages, and use cases — not a one-sentence definition of each format.

You should be able to look at a snippet and name the format; map each format to where Cisco automation actually uses it; and choose a format for a given job (REST body, NETCONF RPC, Ansible inventory, human-edited config).

Format	Typical automation home	What you must notice on sight
JSON	REST / RESTCONF bodies, most modern Cisco APIs	<code>{ }</code> , <code>[]</code> , <code>"keys"</code> , <code>true / false / null</code> , no comments
XML	NETCONF, SOAP, some YANG encodings, device configs	<code><element attr=" ... "></code> , namespaces (<code>xmlns</code>), verbose tags
YAML	Ansible playbooks/inventory, CI files, Kubernetes-style manifests	Indentation, <code>-</code> lists, <code>#</code> comments, unquoted keys

2. Detailed explanation

All three formats serialize the same kind of information: nested maps (key/value), lists, strings, numbers, and booleans. They differ in **syntax**, **strictness**, **human editability**, and **ecosystem**.

JSON (JavaScript Object Notation) is a data interchange format, not a programming language. A JSON document is one value: an object, an array, a string, a number, `true`, `false`, or `null`. Objects are unordered collections of string keys mapped to values. Arrays are ordered lists. Strings use double quotes only. Booleans are lowercase `true` and `false`. Absence of a value is `null`, not `None` or `nil`. JSON has **no comments**. If you see `//` or `#` inside a payload, it is not legal JSON (some parsers tolerate it; the standard does not). JSON is the default for REST APIs because it is compact, maps cleanly onto Python `dict / list`, and is easy to generate from code.

XML (eXtensible Markup Language) is a tagged tree. Every piece of data lives in an **element** (`<name>GigabitEthernet1</name>`) or an **attribute** (`<interface name="GigabitEthernet1">`). Elements can nest. Attributes cannot nest — they are string pairs on a start tag. **Namespaces** (`xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"`) disambiguate vocabularies so two standards can both use an `<interface>` element without colliding. XML is **verbose**: every value is wrapped in a start tag and an end tag. That verbosity is a feature for protocols that need explicit structure, schemas (XSD), and mixed content. In Cisco automation you meet XML primarily through **NETCONF** (RPCs wrapped in XML) and older **SOAP** APIs. RESTCONF can return XML, but JSON is more common in study material.

YAML (YAML Ain't Markup Language) is a human-oriented serialization format. Structure is defined by **indentation** (spaces, never tabs mixed with spaces). Lists use `-`. Mappings use `key: value`. Comments start with `#`. YAML is a **superset of JSON in practice**: a valid JSON document is typically valid YAML, so an Ansible playbook can contain a JSON blob and still parse. The reverse is not true — YAML comments, unquoted strings, and `& / *` anchors are not JSON. Ansible, GitHub Actions, Docker Compose, and many CI systems standardized on YAML because operators edit it by hand.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    subgraph same ["Same logical data"]
        I["interfaces: name, ip, enabled"]
    end
    I --> JSON["JSON: REST / RESTCONF"]
    I --> XML["XML: NETCONF / SOAP"]
    I --> YAML["YAML: Ansible / CI"]
```

Advantages and disadvantages

	JSON	XML	YAML
Compact	High	Low	Medium
Comments	None	<!-- -->	#
Namespaces	No native model	First-class	No native model
Attributes vs elements	N/A (only keys)	Must choose	N/A
Human editing	Easy for small docs	Painful	Designed for it
Strictness	Rigid syntax	Rigid + schemas	Indentation-sensitive
Typical failure	Trailing comma, comments	Namespace miss, unclosed tag	Wrong indent, tab/space mix
Cisco-shaped use	Meraki, Catalyst Center, RESTCONF JSON	NETCONF <rpc> , YANG XML	Ansible playbooks

Use-case rule of thumb: talking to a REST API → JSON. Talking to NETCONF → XML. Writing an Ansible playbook or GitHub workflow → YAML. Do not “convert in your head” by changing quotes; convert by parsing into a data structure, then serializing (objective 1.2).

3. Syntax and examples

The same two interfaces appear in `labs/02_data_formats/` as JSON, YAML, and XML. Study the three files side by side.

JSON object and array

```
{
  "interfaces": [
    {
      "name": "GigabitEthernet1",
      "enabled": true,
      "ipv4": { "address": "10.10.20.48", "prefix": 24 }
    }
  ]
}
```

- `{ }` starts an **object** (Python `dict`).
- `[ ]` starts an **array** (Python `list`).
- Keys are always double-quoted strings.
- `true` is a boolean, not the string `"true"`. `24` is a number, not `"24"`.
- No trailing comma after the last property. No comments.

YAML equivalent (from `labs/02_data_formats/interfaces.yaml`)

```
# YAML allows comments. JSON does not.
interfaces:
  - name: GigabitEthernet1
    enabled: true
    ipv4:
      address: 10.10.20.48
      prefix: 24
```

- The `#` line is a comment.
- `interfaces:` is a mapping key whose value is a list.
- `-` starts a list item. Nested keys under that item are indented **further** (typically 2 spaces).
- `true` and `24` are still boolean and integer after parsing — YAML infers types unless you quote.

XML equivalent (from `labs/02_data_formats/interfaces.xml`)

```
<interfaces xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  <interface>
    <name>GigabitEthernet1</name>
    <enabled>true</enabled>
    <ipv4>
      <address>10.10.20.48</address>
      <prefix>24</prefix>
    </ipv4>
  </interface>
</interfaces>
```

- `xmlns` declares the default **namespace**. Child elements inherit it.

- `<enabled>true</enabled>` is **text**, not a native boolean. Your parser must interpret `"true"` as boolean if you want Python `True`.
- Attributes would look like `<interface name="GigabitEthernet1">`. This sample uses child elements instead — both are legal XML; NETCONF/YANG models usually prefer elements.

Whitespace and quoting traps

Snippet	Format	Verdict
<code>{"enabled": True}</code>	JSON	Invalid. JSON requires <code>true</code> .
<code>{"enabled": true, }</code>	JSON	Invalid. Trailing comma.
<code>enabled: yes</code>	YAML	Often becomes boolean <code>True</code> (YAML 1.1). Quote if you need the string.
<code><enabled>true</enabled></code>	XML	Text node <code>"true"</code> .

4. Exam-style understanding

You should be able to:

1. Identify a blob as JSON, XML, or YAML in one glance (braces vs tags vs indentation).
2. State **one advantage and one disadvantage** of each format relative to the other two.
3. Match a Cisco technology to a format: RESTCONF JSON, NETCONF XML, Ansible YAML.
4. Spot illegal JSON: comments, single quotes, `True` / `None`, trailing commas.
5. Explain why YAML is preferred for Ansible: comments and indentation are easier for humans than JSON braces.

Original practice scenario. An engineer pastes this into a REST client as a JSON body:

```
{
  // management interface
  'name': 'GigabitEthernet1',
  "enabled": True,
}
```

Three independent JSON errors: a `//` comment, single-quoted strings, Python `True`, and a trailing comma. A correct JSON body is `{"name": "GigabitEthernet1", "enabled": true}`. If the same data were an Ansible var file, YAML with a `#` comment would be appropriate.

5. Hands-on exercise

Free tools: a text editor and Python 3. No Cisco gear.

1. Open `labs/02_data_formats/interfaces.json`, `interfaces.yaml`, and `interfaces.xml`.

2. On paper, list every difference you can see: comments, quotes, namespaces, booleans as text vs native, indentation vs tags.
 3. Break JSON on purpose: add a comment, change `true` to `True`, add a trailing comma. Save as `interfaces.bad.json`. Confirm that `python -c "import json; json.load(open('interfaces.bad.json'))"` fails, and read the error.
 4. In YAML, change indentation of `prefix` by one space and run `python parse_formats.py` from `labs/02_data_formats/`. Observe that YAML fails on structure, not on a missing brace.
 5. Write a four-row comparison table in your notes: compactness, comments, Cisco use, typical parse error.
-

1.2 Describe parsing of common data format (XML, JSON, and YAML) to Python data structures

1. What Cisco expects me to know

Describe means explain the concept: what “parse” does, which Python types you get, and which library functions you use. You are not required to memorize every XML XPath feature, but you must know the standard library path for JSON, a safe YAML load, and ElementTree for XML.

Target mapping:

On the wire	After a correct parse in Python
JSON object / YAML mapping	<code>dict</code>
JSON array / YAML list	<code>list</code>
JSON/YAML string	<code>str</code>
JSON/YAML number	<code>int</code> or <code>float</code>
JSON <code>true</code> / <code>false</code>	<code>True</code> / <code>False</code> (<code>bool</code>)
JSON <code>null</code> / YAML <code>null</code>	<code>None</code>
XML element tree	<code>Element</code> objects; you extract text yourself

Official JSON docs: <https://docs.python.org/3/library/json.html>.

2. Detailed explanation

Parsing converts a string (or file bytes) that follows a format’s grammar into in-memory objects your program can index, loop, and branch on. **Serializing** (dumping) is the reverse: Python objects become a string you can send in an HTTP body or write to disk.

Until you parse, `"{\"enabled\": true}"` is just a string. After `json.loads`, it is `{"enabled": True}` and `data["enabled"]` is `True` works.

JSON in the standard library (`json`)

- `json.loads(s)` — parse a **string** (`s` = string).
- `json.load(fp)` — parse an open **file** object.
- `json.dumps(obj)` — serialize a Python object to a **string**.
- `json.dump(obj, fp)` — serialize to a **file**.

Remember the `s`: `loads` / `dumps` work on strings. `load` / `dump` work on files. Mixing them is a common bug (`json.load` on a string raises).

JSON objects become `dict`. Arrays become `list`. Nested structures stay nested. `true` / `false` / `null` become `True` / `False` / `None`. JSON object keys are always strings, so `data[0]` is wrong if the root is an object; `data["interfaces"]` is right.

YAML (`yaml.safe_load`)

PyYAML is not in the standard library; this repo installs it via `labs/requirements.txt`. Use `yaml.safe_load`, not `yaml.load` without a `Loader`. `safe_load` refuses to construct arbitrary Python objects from YAML tags, which would be a code-execution risk. After a safe load, YAML mappings and sequences look like JSON's `dict` and `list`. That is why `parse_formats.py` can compare the three parsed results with `==`.

XML (`xml.etree.ElementTree`)

XML does **not** become a `dict` automatically. `ElementTree` gives you a tree of `Element` nodes. You walk with `.find()`, `.findall()`, `.findtext()`, and you must handle **namespaces**. Text inside an element is a string. There is no automatic `true` → `True`.

Namespaces are the exam-relevant trap. If the document has `xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces"`, then `root.findall("interface")` often returns **nothing**. You search with a Clark notation `{uri}interface` or a prefix map, as the lab does with `NS = {"if": "urn:ietf:params:xml:ns:yang:ietf-interfaces"}` and `"if:interface"`.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    A["Raw string or file"] --> B["Format"]
    B -->|JSON| C["json.loads / json.load"]
    B -->|YAML| D["yaml.safe_load"]
    B -->|XML| E["ET.parse / ET.fromstring"]
    C --> F["dict / list / str / int / bool / None"]
    D --> F
    E --> G["Element tree → extract text → your own dict"]
```

Why this matters for APIs. `response.json()` on a `requests` `Response` is `json.loads` on the body. If you print `response.text` you still have a string. Automation logic (`if iface["enabled"]:`) only works after parsing.

3. Syntax and examples

The teaching implementation is `labs/02_data_formats/parse_formats.py`.

JSON: string vs file

```
import json
from pathlib import Path

text = Path("interfaces.json").read_text(encoding="utf-8")
data = json.loads(text)          # str → dict
# equivalently:
with open("interfaces.json", encoding="utf-8") as f:
    data = json.load(f)          # file → dict

first_name = data["interfaces"][0]["name"] # str
enabled = data["interfaces"][0]["enabled"] # bool
prefix = data["interfaces"][0]["ipv4"]["prefix"] # int

out = json.dumps(data, indent=2) # dict → str (pretty)
```

`data["interfaces"]` is a `list`. Index `0` is a `dict`. Nested `ipv4` is another `dict`. Wrong: `data.interfaces` (that is JavaScript, not Python).

YAML

```
import yaml

with open("interfaces.yaml", encoding="utf-8") as f:
    data = yaml.safe_load(f)
# data["interfaces"][0]["name"] works the same as JSON
```

`safe_load` returns native Python types. Do not use `yaml.load(f)` in automation code.

XML with a namespace

```
from xml.etree import ElementTree as ET

NS = {"if": "urn:ietf:params:xml:ns:yang:ietf-interfaces"}
root = ET.parse("interfaces.xml").getroot()
for iface in root.findall("if:interface", NS):
    name = iface.findtext("if:name", default="", namespaces=NS)
    enabled_text = iface.findtext("if:enabled", default="", namespaces=NS)
    enabled = enabled_text == "true" # convert text → bool
```


`findtext` returns a string or `None`. The lab converts enabled with `== "true"` because XML has no boolean type.

Round-trip check. After parsing all three files, the lab prints `All three agree: True` when the extracted Python lists match. That is the point of parsing: format is a serialization choice; the data structure is what your code owns.

4. Exam-style understanding

You should be able to:

1. Given a JSON string, say which Python type `json.loads` produces at the root (`dict` vs `list`).
2. Choose `loads` vs `load` vs `dumps` vs `dump` for a described task.
3. Explain why `yaml.safe_load` is preferred over `yaml.load`.
4. Explain why an ElementTree `findall("interface")` can return `[]` on a namespaced document.
5. Predict `type(value)` after parse: JSON `null` → `None`; JSON `true` → `bool`; XML `<enabled>true</enabled>` → `str` until you convert.

Original practice scenario. Code does `print(payload["hostname"])` immediately after `r = requests.get(url)`. If `payload = r.text`, that is a string and `payload["hostname"]` is indexing characters, not a key. The fix is `payload = r.json()` (or `json.loads(r.text)`). Same idea as objective 1.2 applied to HTTP (Domain 2).

5. Hands-on exercise

From `labs/02_data_formats/`:

```
python parse_formats.py
```

You should see JSON, YAML, and XML all yield `GigabitEthernet1` and `All three agree: True`.

Then, in a Python REPL:

1. `json.loads('{ "a": true, "b": null }')` — confirm types with `type( ... )`.
 2. `json.dumps({ "enabled": True, "desc": None })` — confirm `true` and `null` on the wire.
 3. Remove the `xmlns` handling in a **copy** of `parse_xml` and watch `findall` return an empty list.
 4. Call `yaml.safe_load` on the contents of `interfaces.json` (JSON as YAML). It should parse. This is the “YAML is a superset of JSON in practice” fact.
-

1.3 Describe the concepts of test-driven development

1. What Cisco expects me to know

Describe the TDD loop and why it exists. Know **red–green–refactor**, that tests are written **before** production code, and that Python’s common tools are **unittest** (standard library) and **pytest** (popular third-party). Domain 4 also asks you to construct a unit test; this objective is the **concept**.

2. Detailed explanation

Test-driven development (TDD) is a design method, not a test-after checklist. You specify behavior with an automated test that **fails**, then write the smallest implementation that makes it **pass**, then **refactor** while the tests stay green.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    R["RED: write a failing test"] --> G["GREEN: write just enough code"]
    G --> F["REFACTOR: clean up, tests stay green"]
    F --> R
```

Red. You write a test for a behavior you want, for example “a /24 has 254 usable hosts.” You run the test runner. It fails because the function does not exist or returns the wrong value. Failure is required: a test that never failed might not be testing anything.

Green. You implement `prefix_to_hosts` until that test passes. You do not add features the tests do not demand.

Refactor. You rename, extract functions, or simplify, **without changing behavior**. The tests are the safety net. If a refactor breaks a test, you broke behavior.

Why network automation cares. Device-facing scripts fail in production in expensive ways (wrong ACL, wrong VLAN). TDD forces you to state expected outputs (254 hosts, /32 → 1) before you talk to a router. You still need integration tests against APIs later; TDD’s unit tests cover pure logic (subnet math, payload shaping, status-code mapping).

unittest vs pytest

	unittest	pytest
Ships with Python	Yes	No (<code>pip install pytest</code>)
Test discovery	Classes subclassing <code>TestCase</code> , methods <code>test_*</code>	Files <code>test_*.py</code> , functions <code>test_*</code>
Assertions	<code>self.assertEqual</code> , <code>self.assertRaises</code>	Plain <code>assert</code>
Exam-adjacent	Matches “construct a unit test” with <code>stdlib</code>	Same idea, less boilerplate

TDD is independent of the runner. The concept is the cycle. The lab uses `unittest` so you have zero extra packages.

What TDD is not. It is not “write tests someday.” It is not 100% coverage theater. It is not a substitute for reading API documentation. It also does not require you to test Cisco’s OS — you test **your** functions.

3. Syntax and examples

The lab `labs/01_python_basics/test_subnet.py` is a TDD artifact. The tests were the specification; `prefix_to_hosts` in `functions_classes_modules.py` is the implementation.

```
import unittest
from functions_classes_modules import prefix_to_hosts

class TestPrefixToHosts(unittest.TestCase):
    def test_slash_24(self):
        self.assertEqual(prefix_to_hosts("192.168.1.0/24"), 254)

    def test_invalid_raises(self):
        with self.assertRaises(ValueError):
            prefix_to_hosts("not-an-ip")
```

- `unittest.TestCase` gives you assertion methods.
- Method names **must** start with `test` or the runner skips them.
- `assertEqual(actual, expected)` fails with a diff if they differ.
- `assertRaises(ValueError)` passes only if that exception is raised.

Run:

```
cd labs\01_python_basics
python -m unittest test_subnet.py
```

A passing run prints `OK` . If you change `254` to `255` , you get `FAIL` and a traceback — that is **red**.

TDD sequence for this lab (do it once on purpose):

1. Rename `prefix_to_hosts` temporarily so import fails → collection error / fail (**red**).

2. Restore the function with `return 0` → tests fail on values (**red**).
3. Implement real math until **green**.
4. Refactor comments or helper names; re-run; still **green**.

4. Exam-style understanding

You should be able to:

1. Name the three phases in order: red, green, refactor.
2. State that tests are written **first**.
3. Recognize `unittest` code: `TestCase`, `test_*`, `assertEqual`.
4. Explain one benefit: specification before implementation; safer refactor; regression detection.
5. Distinguish TDD (process) from a unit test (artifact). You can have unit tests without TDD; TDD always produces tests first.

Original practice scenario. A colleague writes a 400-line script that pushes VLANs, then asks “how do we TDD this?” The honest answer: extract pure functions (validate VLAN ID 1–4094, build the JSON body), TDD those, and keep the HTTP call thin. You do not TDD the network itself with `unittest` alone.

5. Hands-on exercise

1. Run `python -m unittest test_subnet.py` from `labs/01_python_basics/`. Confirm **OK**.
2. Change `test_slash_24` expected value to `255`, run again, read the failure (**red**).
3. Revert the test. In `prefix_to_hosts`, temporarily `return 254` for every prefix. See which tests fail (`/30`, `/32`) — this shows why **multiple tests** exist.
4. Add a new test `test_slash_31` for `10.0.0.0/31` **before** you look at the implementation. Run (**red** or **green** depending on the lab’s `/31` rule). Align your understanding with the docstring: `/31` and `/32` return `num_addresses` with `no - 2`.
5. Optional: `pip install pytest` and run `pytest test_subnet.py` — `pytest` can run `unittest` tests. Same cycle, different runner.

1.4 Compare software development methods (agile, lean, and waterfall)

1. What Cisco expects me to know

Compare agile, lean, and waterfall: how work is sequenced, how requirements change is treated, where documentation sits, and which situations fit each method. Cisco automation programs in enterprises often **say** Agile while still having waterfall-like change windows. You need the textbook comparison plus that operational reality.

2. Detailed explanation

A **software development method** (or lifecycle model) is a policy for **when** you gather requirements, design, code, test, and release. The exam names three: waterfall, agile, and lean.

Waterfall is sequential. You finish requirements, then design, then implementation, then testing, then operations. You do not (in the pure model) go back. Documentation is heavy because the next phase inherits a frozen spec. Change is expensive: a missed VLAN requirement discovered in testing means revisiting design. Waterfall fits **well-understood, highly regulated, hard-to-reverse** work (a datacenter cutover with a fixed maintenance window and a signed design). It fails when the customer cannot know the full API surface on day one.

Agile is iterative and incremental. Work is split into short cycles (often called sprints). Each cycle aims to produce **working software** (a script that actually creates a Webex room, not a 40-page design that nobody ran). Requirements are expected to change. Collaboration with the “customer” (the network team consuming the automation) is continuous. Agile is a family (Scrum, Kanban, XP); the exam-level idea is iteration, feedback, and adapting scope. Ceremony (stand-ups, boards) is a means, not the definition.

Lean comes from manufacturing (Toyota) and focuses on **eliminating waste**: unused features, waiting, handoffs, defects, extra inventory (including “inventory” of unfinished Git branches). Lean prefers small batches, fast flow, and measuring value. Kanban boards, limiting work-in-progress, and “stop the line” on defects are lean-flavored. Agile and lean overlap; lean is the waste/flow lens, agile is the iterative-delivery lens.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph Waterfall
        W1[Requirements] --> W2[Design] --> W3[Implement] --> W4[Test] --> W5[Operate]
    end
    subgraph Agile
        A1[Sprint N: plan] --> A2[Build a slice] --> A3[Demo / feedback] --> A1
    end
    subgraph Lean
        L1[See the value stream] --> L2[Cut waste] --> L3[Smaller batch] --> L1
    end
```

Dimension	Waterfall	Agile	Lean
Sequence	Linear phases	Repeated short cycles	Continuous flow
Requirements	Frozen early	Expected to change	Pull only what creates value
Documentation	Heavy, phase-gated	“Working software over comprehensive docs” (not zero docs)	Docs that do not add value are waste
Feedback	Late (after test phase)	Every iteration	Continuous; defects stop the line
Change cost	High late	Absorbed in backlog	Reduce by smaller batches
Risk	Late integration surprises	Scope may churn	Starvation if WIP is unmanaged
Fits	Fixed contract, compliance, known design	Evolving APIs, automation platforms	Pipeline efficiency, ops + dev flow
Pitfall	Months of design, untested assumptions	“Agile” as no planning	Cutting “waste” that was actually compliance

Advantages / disadvantages (say them out loud)

- Waterfall advantage: predictable milestones and audit trail. Disadvantage: late discovery of wrong assumptions.
- Agile advantage: early working automation and course correction. Disadvantage: can thrash if the product owner never says no; needs discipline.
- Lean advantage: shorter lead time, less unfinished work. Disadvantage: easy to misread “waste” and delete necessary review or testing.

In network automation, pushing a routing change is often **waterfall-like** (CAB, change window), while **building** the Python/Ansible that will do it can be agile (weekly stories: parse inventory, then dry-run, then apply). Lean shows up in CI: do not pile 40 unreviewed PRs.

3. Syntax and examples

These methods are not code syntax. The “syntax” is how you would **plan** the same automation job three ways.

Job: “Add a REST script that sets hostname on 50 IOS XE boxes via RESTCONF.”

Waterfall plan

1. Requirements document: auth method, YANG path, rollback, success criteria.
2. Design: sequence diagram, error matrix, all 50 hostnames.

3. Implement the entire script.
4. Test in a lab, then a maintenance window.
5. Operate / hand off.

You would not demo a half-script to stakeholders mid-phase.

Agile plan

- Sprint 1: one Always-On DevNet box, GET hostname, show JSON. Demo.
- Sprint 2: POST/PATCH hostname on one box with dry-run logging. Demo.
- Sprint 3: loop inventory YAML, add unittest for payload builder, add 401/404 handling.
- Backlog can insert “use token instead of basic auth” when security asks.

Lean plan

- Map the value stream: request → Git PR → CI → sandbox → production.
- Waste: waiting two weeks for a shared sandbox; re-typing hostnames; merging only once a month.
- Change: smaller PRs, automated tests (`test_subnet.py` style for payload functions), pull from a Kanban column with WIP limit 2.

A GitHub Actions file (see `labs/04_git/workflow.yaml` as a checklist of Git operations, not a full CI engine) is a **lean/agile artifact**: automate checks so waiting on a human to “remember to run unittest” is not in the path.

4. Exam-style understanding

You should be able to:

1. Identify a process description as waterfall (phases, freeze, test at end), agile (sprints, working increment, changing requirements), or lean (waste, small batch, flow).
2. Give one advantage and one disadvantage for each.
3. Pick a method for a scenario: nuclear change window vs evolving internal API vs reducing ticket cycle time.
4. Avoid the trap “Agile has no documentation” — Agile de-emphasizes **comprehensive up-front** docs, not runbooks.
5. Avoid the trap “Lean is just Agile” — related, different emphasis.

Original practice scenario. A team spends six months writing a “fully complete” NSO service, never loads it on a device, then discovers the YANG model in production differs. That is waterfall risk (late feedback). An agile team would have applied a thin service to a sandbox in week two. A lean reading: the inventory of untested design was waste.

5. Hands-on exercise

Free: a whiteboard or markdown file. No paid ALM tool required. A GitHub project board on a free account is enough if you want a board.

1. Write the same three-bullet plan for **your** next lab (for example Domain 2 `rest_client.py`) in waterfall, agile, and lean form.
2. Time-box an agile slice: 45 minutes to get `GET https://httpbin.org/get` working and committed, then stop. That is an increment.
3. Lean audit: list waste in your study setup (reinstalling packages globally instead of a venv, committing `.env`, never running tests). Remove one waste item.
4. Explain to a peer (or a voice note) why a firmware upgrade on a core pair is often waterfall while a Meraki dashboard script is often agile.

1.5 Explain the benefits of organizing code into methods / functions, classes, and modules

1. What Cisco expects me to know

Explain the benefits — not “define function.” Why split a script? What does a class buy you? What is a module? Cisco’s wording uses **methods / functions, classes, and modules**. In Python, a **function** is `def`; a **method** is a function on a class (`self`); a **module** is a `.py` file you import.

2. Detailed explanation

A 300-line `automate.py` that mixes HTTP, JSON parsing, VLAN math, and print statements will work once. It will not work for the exam’s mental model or for a team.

Functions (and methods) name a unit of work with inputs and outputs. Benefits:

- **Reuse:** `prefix_to_hosts("10.0.0.0/30")` from tests, CLI, and a REST wrapper without copy-paste.
- **Testability:** TDD (1.3) needs a callable. You cannot unit-test “lines 80–140 of a script” cleanly.
- **Readability:** `build_inventory(raw)` states intent; a nested loop of dict unpacking does not.
- **Scope:** Local variables stay local. You stop colliding on `data =`.
- **Single responsibility:** One function parses; another posts; another logs. Failures have a smaller blast radius.

A **method** is a function bound to an object. `iface.shutdown()` mutates that interface’s `enabled` flag. The benefit over a loose function `shutdown(iface)` is encapsulation: the object carries its data and the operations that are legal on it.

Classes group **state + behavior**. Benefits:

- Model domain objects (`Interface` , `Device` , `Prefix`) the way the network already thinks.
- Invariants live in one place (`__init__` validates name).
- Multiple instances: 50 interfaces without 50 global variables.
- Foundation for patterns in 1.6 (Model in MVC is typically a class).

You do not need inheritance trees for this exam. A small class with `__init__` and two methods is enough.

Modules are files (and packages are directories of files). Benefits:

- **Namespace:** `from functions_classes_modules import prefix_to_hosts` vs pasting the function into every script.
- **Reuse across programs:** `parse_formats.py` functions could be imported by a RESTCONF tool later.
- **Team parallel work:** one module per concern (http, inventory, tests).
- `if __name__ == "__main__":` the file is both a library and a runnable demo. Tests import it without executing the demo prints.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph TB
        M["module: functions_classes_modules.py"]
        F1["function: prefix_to_hosts"]
        C["class: Interface"]
        MD["methods: shutdown, summary"]
        F2["function: build_inventory"]
        T["module: test_subnet.py"]
        F3[" "]

        M --> F1
        M --> C
        C --> MD
        M --> F2
        T --> F3
    end
```

Disadvantage of not organizing: copy-paste drift (one file subtracts 2 hosts on `/32` , another does not), untestable scripts, import cycles if you split badly, and “god classes.” The benefit of organization is not extra files for their own sake; it is **boundaries**.

3. Syntax and examples

Lab: `labs/01_python_basics/functions_classes_modules.py` .

Function

```
from ipaddress import ip_network

def prefix_to_hosts(prefix: str) → int:
    net = ip_network(prefix, strict=False)
    if net.prefixlen ≥ 31:
        return net.num_addresses
    return net.num_addresses - 2
```

- `def` names the function. Parameters are inputs. `return` is the output.
- Call site: `prefix_to_hosts("192.168.10.0/24")` → 254.
- Benefit on display: `/31` and `/32` special cases live in **one** place.

Class and methods

```
class Interface:
    def __init__(self, name: str, ip: str, enabled: bool = True):
        self.name = name
        self.ip = ip
        self.enabled = enabled

    def shutdown(self) → None:
        self.enabled = False

    def summary(self) → dict:
        return {"name": self.name, "ip": self.ip, "enabled": self.enabled}
```

- `__init__` is the constructor. `self` is the instance.
- `shutdown` is a **method**: it uses instance state.
- `summary` returns a `dict` ready to become JSON (1.1 / 2.9).

Module import

```
from functions_classes_modules import prefix_to_hosts
```

`test_subnet.py` imports the function — that is the module benefit. The demo under `if __name__ == "__main__":` does not run during `unittest`.

Factory function using the class

```
def build_inventory(raw: list[dict]) → list[Interface]:
    return [Interface(**item) for item in raw]
```

`**item` unpacks dict keys into constructor arguments. One function converts API-like JSON lists into objects.

4. Exam-style understanding

You should be able to:

1. List benefits: reuse, testing, readability, encapsulation, namespace.
2. Distinguish function vs method vs module in a snippet.
3. Explain why tests import a module instead of exec'ing a script.
4. Recognize that a class is appropriate when you have **multiple instances with the same operations**.

5. Reject “modules are only for huge programs” — even two files (`functions_classes_modules.py` + `test_subnet.py`) justify a module.

Original practice scenario. Two scripts both contain a copied `prefix_to_hosts`. A bugfix lands in one. Production uses the other. Organizing into a module with tests would have made the fix universal and verified.

5. Hands-on exercise

```
cd labs\01_python_basics
python functions_classes_modules.py
python -m unittest test_subnet.py
```

Then:

1. Add a method `no_shutdown` that sets `enabled = True`. Call it from the `__main__` block.
2. Move `prefix_to_hosts` into a new file `subnetmath.py` and fix the import in the test. You just created a second module.
3. Note what broke if you forgot to update the import — that is the module boundary teaching you its job.

1.6 Explain the advantages of common design patterns (MVC and Observer)

1. What Cisco expects me to know

Explain the advantages of MVC and Observer. You do not need the Gang of Four book. You need: what problem each pattern solves, how the parts collaborate, and why automation/API systems use them.

Webhooks and event notifications are Observer in network clothing — Domain 2.2 will reuse this.

2. Detailed explanation

A **design pattern** is a reusable arrangement of types and message flow. Two patterns appear on the blueprint.

MVC — Model–View–Controller

- **Model:** data and rules. In a web GUI for switches, the Model is the interface inventory, VLAN membership, “enabled” flags — possibly classes like `Interface`. The Model does not know about HTML or REST URL paths.
- **View:** presentation. A table of interfaces in a browser, a CLI table, or a JSON pretty-print. The View should not contain subnet math.
- **Controller:** input handling. HTTP request arrives; Controller parses parameters, calls the Model, picks a View. In a REST API, the “controller” is the route handler that maps `GET /interfaces` to `inventory.list()`.

Advantage of MVC: you can change the View (Webex Adaptive Card vs HTML vs JSON API) without rewriting business rules. You can test the Model with `unittest` without a browser. Multiple Controllers (REST, CLI, chatbot) can share one Model. The cost of mixing all three is the classic “script that prints while it mutates while it parses.”

In Cisco’s world, a dashboard (Catalyst Center UI) is a View; the device inventory is the Model; REST endpoints are Controllers for programmatic Views (`application/json`).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    U[User or API client] --> C[Controller]
    C --> M[Model]
    M --> C
    C --> V[View]
    V --> U
```

Observer

A **subject** (publisher) maintains a list of **observers** (subscribers). When the subject’s state changes, it **notifies** observers. Observers do not poll in a tight loop. They react.

Advantage of Observer: loose coupling. The switch does not know what will happen when an interface goes down — a syslog collector, a webhook to Webex, a ticket bot. You add observers without editing the subject’s core logic. That is how **webhooks** work: you subscribe a URL; the platform POSTs when an event occurs (2.2). MQTT, syslog, and “event-driven Ansible” are the same idea.

Advantage vs polling: less delay, less wasted GET traffic, but you must handle retries, authentication of the callback, and failure of observers (the subject should not crash if one observer is down).

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    S[Subject: interface state] -->|notify| O1[Observer: logging module]
    S -->|notify| O2[Observer: webhook to https://hooks.example/ccna]
    S -->|notify| O3[Observer: metrics]
```

MVC vs Observer. MVC structures **layers** of an application. Observer structures **notifications**. A Controller might **observe** a Model and refresh a View — the patterns compose.

3. Syntax and examples

MVC sketched in the same objects as the lab

```

# Model
class Interface:
    def __init__(self, name, enabled=True):
        self.name = name
        self.enabled = enabled

# Controller (would be a Flask route or CLI command)
def shutdown_controller(inventory, name):
    iface = next(i for i in inventory if i.name == name)
    iface.shutdown()
    return iface.summary() # data for the View

# View
def print_view(summary: dict) → None:
    print(f"{summary['name']}: enabled={summary['enabled']}")

```

Advantage on display: `print_view` can be replaced by `json.dumps` for an API without touching `Interface.shutdown`.

Observer sketched

```

class InterfaceSubject:
    def __init__(self, name):
        self.name = name
        self.enabled = True
        self._observers = []

    def attach(self, fn):
        self._observers.append(fn)

    def shutdown(self):
        self.enabled = False
        for fn in self._observers:
            fn(self.name, "down")

def webhook_observer(name, state):
    # later: requests.post(webhook_url, json={ ... })
    print(f"POST event {name}={state}")

gil = InterfaceSubject("GigabitEthernet1")
gil.attach(webhook_observer)
gil.shutdown()

```

The subject does not import Webex or ServiceNow. Observers register. That is the advantage.

`labs/01_python_basics/functions_classes_modules.py` comments `Interface` as a “minimal model” — keep that mapping in your head when you reach webhooks.

4. Exam-style understanding

You should be able to:

1. Name MVC's three parts and one responsibility each.
2. Give MVC's advantage: separation of data, presentation, and input; easier testing and multiple UIs.
3. Name Observer's parts: subject, observers, notify on change.
4. Give Observer's advantage: loose coupling, event-driven updates, add subscribers without changing the core.
5. Connect Observer → webhooks (HTTP callback on event) and MVC → web/API apps.

Original practice scenario. A script every 30 seconds GETs `/alarms` (polling). An Observer design would register a webhook so the controller POSTs when an alarm opens. Advantage: faster notice, fewer GETs. New constraint: you must expose a URL and validate the sender (Domain 2).

5. Hands-on exercise

1. Read `Interface` in `labs/01_python_basics/functions_classes_modules.py`. Label Model vs what a View/Controller would be.
2. Add a list of callbacks to `Interface.shutdown` that print a line (mini-Observer). Do not touch `summary`.
3. Draw the mermaid MVC and Observer diagrams from memory.
4. When you study 2.2, return here and rewrite `webhook_observer` using `requests.post` to <https://httpbin.org/post>.

1.7 Explain the advantages of version control

1. What Cisco expects me to know

Explain the advantages of version control as a practice. Git is the tool (1.8). This objective is **why** you use it: history, collaboration, branching, rollback, audit, review. "We have a shared USB stick called FINAL_v3" is the anti-pattern.

2. Detailed explanation

Version control records snapshots of files over time, with authorship, messages, and the ability to branch parallel lines of work. Git is a **distributed** version control system: every clone has the full history (1.8.a).

Advantages

Advantage	What it means in automation
History	Who changed the ACL template and when
Rollback	Last week's playbook still exists; <code>git revert</code> / checkout an old commit
Branching	Try RESTCONF auth changes without breaking <code>main</code>
Collaboration	Two engineers, one repo, merge (1.8.f) instead of emailing zips
Review	Pull requests: a second person sees a <code>diff</code> (1.8.g) before production
Audit / compliance	Evidence of what was deployed; tags for releases
Backup	The remote (GitHub) is not the only copy, but it is a durable one
Bisect / debug	Binary search history to find the commit that broke parsing
Automation of automation	CI runs tests on each commit (TDD artifacts stay green)

What version control is not. It is not a backup substitute if nobody pushes. It is not a secrets manager — **do not commit API keys** (see 2.7). It is not a substitute for testing.

Centralized vs distributed (context). Older systems (central SVN) required the server for many operations. Git lets you commit on an airplane, then push. The exam cares that you understand **local vs remote** (1.8), which exists because Git is distributed.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart LR
    WT[Working tree] --> ST[Staging index]
    ST --> LR[Local repository]
    LR -->|push| RM[Remote repository]
    RM -->|pull / fetch| LR

```

Advantages accrue only if the team actually **commits small, meaningful snapshots** and uses branches for isolation. A single commit named “stuff” with 40 unrelated files throws away history’s value.

3. Syntax and examples

Version control’s “syntax” is the **commit**, the **message**, and the **log**.

```

git log --oneline
# a1b2c3d Add RESTCONF hostname helper
# 9f8e7d6 Fix YAML indent in inventory

```

Each hash is a complete snapshot pointer. Advantage: you can describe `git show a1b2c3d` in a change ticket.

`.gitignore` is part of using version control well:

```
.venv/  
labs/.env  
__pycache__/
```

Advantage: history stays about **source**, not secrets and byte-compiled junk.

`labs/04_git/workflow.yaml` lists the operations you will **utilize** in 1.8. The advantage list in 1.7 is why that workflow exists.

Bad vs good commit (advantage of discipline)

Message	History advantage
update	Near zero
Fix prefix_to_hosts /31 usable-host count	Future you can find the bugfix

4. Exam-style understanding

You should be able to:

- 1. List several advantages: history, collaboration, branch/merge, rollback, review, audit.
- 2. Contrast version control with “copy the folder to dated names.”
- 3. State that Git being distributed means commits happen **locally** even before push.
- 4. State that secrets in Git are a disadvantage of **misuse**, not of version control itself.
- 5. Connect advantages to operations: rollback ↔ commit history; collaboration ↔ push/pull; review ↔ diff.

Original practice scenario. Production Ansible used `enabled: true` for a shutdown window because an intern edited live on the server. With version control, the change would have been a branch, a diff, and a review; rollback would be a revert, not “does anyone have last Tuesday’s file?”

5. Hands-on exercise

Free: Git and a **private** GitHub repo (<https://github.com/>). Official Git docs: <https://git-scm.com/docs>.

- 1. Read `labs/04_git/workflow.yaml` end to end.
- 2. Initialize or clone a throwaway private repo (do **not** publish `labs/.env`).
- 3. Make three tiny commits with messages that a stranger could understand. Run `git log`.
- 4. Write five advantages in your own words without looking at the table above.

5. Add a `.gitignore` that excludes `.env` and `.venv`. Confirm `git status` does not list them.

1.8 Utilize common version control operations with Git

1. What Cisco expects me to know

Utilize means **perform** the operations, not only define them. The sub-objectives are clone, add/remove, commit, push/pull, branch, merge and handling conflicts, and diff. This parent section is the mental model those commands sit on. Official command reference: <https://git-scm.com/docs>.

2. Detailed explanation

Git has **four places** people confuse:

Place	What it is	Typical command that touches it
Working tree	Files you edit	editing in VS Code
Staging area (index)	The next commit's proposed snapshot	<code>git add</code> , <code>git rm</code>
Local repository	Commits on your machine (<code>.git</code>)	<code>git commit</code>
Remote repository	Copy on GitHub/GitLab	<code>git push</code> , <code>git pull</code> , <code>git clone</code>

Clone copies a remote (including history) to a new local repo. **Add** copies working-tree changes into the index. **Commit** records the index as a snapshot with a message. **Push** sends local commits to a remote. **Pull** fetches remote commits and integrates them (merge or rebase; this course uses merge). **Branch** is a movable pointer to a commit — cheap parallel lines of development. **Merge** combines histories; **conflicts** happen when both sides changed the same region. **Diff** shows line-level changes.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    subgraph local ["Your machine"]
        WT[Working tree]
        IX[Staging index]
        REPO[(Local commits)]
        WT -->|git add| IX
        IX -->|git commit| REPO
        REPO -->|git checkout / switch| WT
    end
    REMOTE[(origin on GitHub)]
    REMOTE -->|git clone / git pull| REPO
    REPO -->|git push| REMOTE
```

`git status` is the orientation command: it tells you which of those places has uncommitted or unpushed work.

3. Syntax and examples

Identity (once per workstation; you already have this in `CCNAAUTO_LAB_SETUP.md`):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Orientation:

```
git status
git log --oneline --decorate --graph --all
```

The detailed syntax for each operation is in 1.8.a–1.8.g. A full loop looks like:

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git switch -c feature/hostname
# edit files
git add hostname.py
git commit -m "Parameterize RESTCONF base URL"
git push -u origin feature/hostname
# later, on main:
git pull
git merge feature/hostname
git diff HEAD~1
```

4. Exam-style understanding

You should be able to:

1. Place `add`, `commit`, `push` on the working tree → index → local → remote path.
2. Know `clone` creates the local repo from a remote; it is not the same as `commit`.
3. Know `pull` is not automatic just because files changed on GitHub — you must pull (or fetch).
4. Know a conflict is not a Git crash; it is a merge that needs a human.
5. Read a unified diff (1.8.g, lab file `labs/04_git/example.diff`).

Practice item: “I edited `notes.txt` but `git log` does not show it.” Cause: no add/commit. “I committed but GitHub does not show it.” Cause: no push.

5. Hands-on exercise

Use a **private** GitHub repository and the checklist in `labs/04_git/workflow.yaml`. Follow the clone-through-diff sequence in `CCNAAUTO_LAB_SETUP.md` section “Git lab (objectives 1.8.a–g and 5.12)”. Study `labs/04_git/example.diff` until you can explain every line. The subsections below are the graded skills; do each hands-on block, not only this parent one.

1.8.a Clone

1. What Cisco expects me to know

Utilize clone: create a local copy of a repository from a remote URL, including history and a link to `origin`.

2. Detailed explanation

`git clone <url>` is how you **start** from an existing project (Cisco samples on GitHub, your `ccnaauto-labs` repo, a teammate’s library). Clone:

1. Creates a new directory (by default named after the repo).
2. Copies all reachable commits (full history in a normal clone).
3. Checks out the default branch (`main` or `master`) into the working tree.
4. Sets `remote.origin.url` so later `push` / `pull` know where to go.

Clone is **not** download-zip. A zip has files but no `.git` history, no branches, no remotes. Clone is **not** `git init` in a random folder — `init` creates an empty repo; clone fills it from elsewhere.

HTTPS vs SSH URLs both work. DevNet Code Exchange examples are often HTTPS:

```
https://github.com/CiscoDevNet/ ... .
```

3. Syntax and examples

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git remote -v
git branch -vv
```

- First argument is the URL.
- Optional second argument is the directory name: `git clone <url> lab-copy`.
- `git remote -v` should show `origin` fetch and push URLs.
- Shallow clone `git clone --depth 1 <url>` is a real Git feature for speed; you still cloned, but history is truncated. Prefer a full clone for study.

4. Exam-style understanding

You should be able to:

1. State the purpose: copy a remote repo locally, including Git metadata.
2. Recognize that after clone you already have a local `main` and a remote `origin`.
3. Distinguish clone (new directory from remote) vs pull (update an existing clone).
4. Know that clone of a private repo requires authentication (GitHub login/token).

Original practice: “Clone the repo then immediately `git commit`.” That commit needs a change; clone itself does not create a new commit.

5. Hands-on exercise

```
git clone https://github.com/<you>/ccnaauto-labs.git
cd ccnaauto-labs
git status
git log --oneline -5
```

If you do not have a GitHub repo yet, create a private empty one in the browser, clone it, and proceed with 1.8.b. Do not clone secrets-filled repos.

1.8.b Add/remove

1. What Cisco expects me to know

Utilize add and remove: stage new/modified files (`git add`) and stage a file’s deletion (`git rm`). The index is the next commit.

2. Detailed explanation

Editing a file only changes the **working tree**. Git still thinks the last commit is the truth until you **stage** and **commit**.

- `git add <path>` stages a new file or a modification.
- `git add -A` or `git add .` stages all changes in the repo or current directory (use with care).
- `git rm <path>` deletes the file from the working tree **and** stages the deletion.
- Deleting in Explorer/VS Code without `git rm` leaves a “deleted” working-tree change; you still `git add` that deletion (or `git add -u`) to stage it.
- `git restore --staged <path>` (or older `git reset HEAD <path>`) **unstages** without destroying the file.

Remove vs ignore. `git rm` is for files that **were** tracked. `.gitignore` prevents **untracked** files from being added. If a secret was already committed, gitignore will not un-publish it; you must stop using the secret.

3. Syntax and examples

```
echo "lab" > notes.txt
git add notes.txt
git status
# Changes to be committed: new file notes.txt

git rm notes.txt
git status
# Changes to be committed: deleted notes.txt
```

Partial add:

```
git add labs/02_data_formats/parse_formats.py
# does not stage unrelated dirty files
```

Windows PowerShell: `git add` paths can use `/` or `\`; Git typically stores paths with `/`.

4. Exam-style understanding

You should be able to:

1. Explain that add copies content into the **staging area**.
2. Choose `git rm` when the intent is “this file should disappear in the next commit.”
3. Read `git status` sections: untracked vs changes not staged vs changes to be committed.
4. Avoid thinking `git add` sends files to GitHub (that is push).

Original practice: `git add .` from the repo root after creating `labs/.env` — disaster if `.gitignore` is missing. Add/remove skill includes **what not to add**.

5. Hands-on exercise

In your clone:

1. Create `notes.txt`, `git add notes.txt`, `git status`.
 2. Edit `notes.txt`, `git status` (modified, not staged), `git add notes.txt` again.
 3. `git rm notes.txt` **after** it has been committed at least once (commit first in 1.8.c if needed), or practice `git rm --cached` on a dummy file to untrack without deleting locally — then restore cleanly.
 4. Confirm `labs/.env` would be ignored if you copy the course `.gitignore`.
-

1.8.c Commit

1. What Cisco expects me to know

Utilize commit: record a staged snapshot in the **local** repository with a message. Commit does not update GitHub by itself.

2. Detailed explanation

A commit object stores: tree (file snapshot), parent commit(s), author, timestamp, message. The current branch pointer moves to the new commit.

Rules that preserve the advantages from 1.7:

- Stage only related changes.
- Write a message that says **why**, not “files.”
- Commits are local until push. You can have many local commits.

`git commit -m "message"` uses the message flag. `git commit` without `-m` opens an editor. `git commit -a` stages tracked modifications and commits (still will not add **new** untracked files). Prefer explicit `git add` while learning.

Amending (`git commit --amend`) rewrites the last commit. Do not amend commits already pushed to a shared branch unless you know the consequences; this study guide does not require amend.

3. Syntax and examples

```
git add notes.txt
git commit -m "Add lab notes"
git log -1
```

Good messages for this course:

```
Add prefix_to_hosts unit tests
Parameterize RESTCONF hostname URL
```

Empty commit attempt: if nothing is staged, Git refuses (`nothing to commit`). That is a feature.

4. Exam-style understanding

You should be able to:

1. State that commit records the **index** to the **local** repo.
2. Know a message is required (policy) and `-m` supplies it.

3. Sequence: edit → add → commit → (later) push.
4. Recognize `nothing to commit, working tree clean`.

Original practice: “I committed, why is the GitHub file unchanged?” Because commit is local.

5. Hands-on exercise

```
echo "lab" > notes.txt
git add notes.txt
git commit -m "Add lab notes"
git status
git log --oneline -3
```

Make a second commit that only changes one line. Confirm two hashes in `git log`.

1.8.d Push / pull

1. What Cisco expects me to know

Utilize push and pull: publish local commits to a remote; bring remote commits into your local branch.

2. Detailed explanation

Push (`git push`) uploads your commits to `origin` (usually). The first push of a branch often needs `-u origin <branch>` to set upstream so later `git push` has a default.

Pull (`git pull`) is `git fetch` (download remote commits) plus an integrate step. Default integrate is **merge** if you set `pull.rebase false` as in the lab setup. Pull is how you sync before starting work and how you receive a teammate’s commits.

If the remote has commits you lack, push is rejected until you pull (or otherwise reconcile). That protection prevents silently overwriting published history.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant L as Local main
    participant O as origin/main
    L->>O: git push
    O->>L: git pull
```

Fetch vs pull. `git fetch` updates remote-tracking branches (`origin/main`) without merging. `git pull` fetches and merges into your current branch. For the exam, pull = get remote changes into my branch.

3. Syntax and examples

```
git push -u origin main
git push
git pull
```

New branch:

```
git switch -c feature/hostname
git push -u origin feature/hostname
```

If push fails with “non-fast-forward”:

```
git pull
# resolve conflicts if any (1.8.f)
git push
```

Authentication: GitHub HTTPS may require a personal access token instead of a password. SSH uses keys. The exam tests the **operations**, not GitHub’s current login UI.

4. Exam-style understanding

You should be able to:

1. Push = local commits → remote. Pull = remote commits → local branch.
2. Clone already sets `origin` ; push/pull use it.
3. Explain a rejected push: remote moved, you must pull first.
4. Know `-u` sets the upstream tracking branch.

Original practice: Two clones of the same repo. Commit in A, push. Clone B still has old files until `git pull` in B.

5. Hands-on exercise

```
git push -u origin main
# on GitHub, confirm the commit
# from a second folder:
git clone https://github.com/<you>/ccnaauto-labs.git ccnaauto-labs-b
cd ccnaauto-labs-b
# make a commit on GitHub in the browser or in clone A, then:
git pull
```

Practice both directions once.

1.8.e Branch

1. What Cisco expects me to know

Utilize branch: create, list, and switch branches so work is isolated from `main`.

2. Detailed explanation

A **branch** is a pointer to a commit. Creating a branch is cheap (a 41-byte ref, not a full copy of files). When you commit, the **current** branch pointer moves forward.

Why branch: keep `main` releasable; develop a RESTCONF change without mixing it into an unrelated YAML fix; enable pull-request review.

Switch vs checkout. Modern Git: `git switch <name>` to change branches, `git switch -c <name>` to create and switch. Older: `git checkout -b <name>`. Both appear in the wild.

You cannot switch with uncommitted conflicting changes (Git will stop you). Commit or stash first. Stash is useful but not a named 1.8 sub-objective; prefer commit on the feature branch.

3. Syntax and examples

```
git branch          # list local; * marks current
git switch -c feature/hostname
git switch main
git branch -d feature/hostname # delete after merge; -D forces
```

`git branch -vv` shows tracking remotes.

Naming: `feature/hostname`, `fix/yaml-indent`. Slashes are allowed in branch names.

4. Exam-style understanding

You should be able to:

1. Define a branch as a movable pointer to a commit.
2. Create and switch with `git switch -c` or `git checkout -b`.
3. Explain the advantage: isolated history until merge.
4. Read `git status` "On branch feature/hostname".

Original practice: Commits while on `feature/hostname` do not move `main` until merge.

5. Hands-on exercise

```
git switch -c feature/hostname
# edit a file, add, commit
git switch main
git log --oneline --graph --all
```

Confirm `main` does not include the new commit until 1.8.f.

1.8.f Merge and handling conflicts

1. What Cisco expects me to know

Utilize merge and **handle conflicts**: combine a branch into another; when both sides edit the same lines, resolve markers, then commit.

2. Detailed explanation

Merge creates a new commit (except fast-forward) that has **two parents** and a tree combining both histories.

Fast-forward: `main` has not moved since you branched; Git just slides `main` to the feature tip. No merge commit.

True merge: `main` gained commits while you worked; Git weaves them.

Conflict: Git cannot automatically combine a hunk. It writes **conflict markers** into the file:

```
<<<<<<< HEAD
url = "https://router/restconf/data/Cisco-IOS-XE-native:native/hostname"
=====
HOST = "https://router"
url = f"{HOST}/restconf/data/Cisco-IOS-XE-native:native/hostname"
>>>>>>> feature/hostname
```

`HEAD` is “the branch you merged **into**” (often `main`). The other label is the incoming branch. You **edit** to the correct final content, **remove** the markers, `git add` the file, and `git commit` to complete the merge.

Abort: `git merge --abort` if you want to undo an in-progress merge.

Do not leave markers in production files. That is a failed resolution.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    M[main] --> C[merge commit]
    F[feature/hostname] --> C
```

3. Syntax and examples

Clean merge:

```
git switch main
git merge feature/hostname
```

Force a conflict (lab):

```
git switch main
echo "alpha" > conflict.txt
git add conflict.txt && git commit -m "Main says alpha"
git switch -c feature/conflict
echo "bravo" > conflict.txt
git add conflict.txt && git commit -m "Feature says bravo"
git switch main
echo "charlie" > conflict.txt
git add conflict.txt && git commit -m "Main now charlie"
git merge feature/conflict
# conflict.txt contains markers
```

Resolve by choosing the correct content (or combining), then:

```
git add conflict.txt
git commit -m "Merge feature/conflict; keep charlie"
```

`git status` during a conflict says “You have unmerged paths.”

4. Exam-style understanding

You should be able to:

1. State merge integrates two branches.
2. Recognize `<<<<<<`, `====`, `>>>>>>`.
3. Describe the fix: edit, add, commit.
4. Know fast-forward vs merge commit at a high level.
5. Know that both sides changing **different** files usually merges cleanly.

Original practice: Same line, two branches, merge, conflict. Different lines in the same file often auto-merge.

5. Hands-on exercise

Perform the conflict sequence above in `ccnaauto-labs`. Then merge a **non-conflicting** `feature/hostname` as in the lab setup. Read `git log --graph --oneline` after both.

1.8.g diff

1. What Cisco expects me to know

Utilize diff: view unstaged, staged, and commit-to-commit changes. Read **unified diff** (headers, hunks, `+/−`). This also supports exam topic 5.12-style interpretation later.

2. Detailed explanation

A **diff** is a line-oriented comparison. Git's default unified format shows enough context to apply a patch.

- `git diff` — working tree vs index (unstaged).
- `git diff --staged` (`--cached`) — index vs last commit (what the next commit will contain).
- `git diff HEAD~1` — last commit vs current HEAD.
- `git diff main ... feature/hostname` — changes on the feature since it diverged.

Reading the format (see `labs/04_git/example.diff`):

Piece	Meaning
<code>--- a/hostname.py</code>	Old file
<code>+++ b/hostname.py</code>	New file
<code>@@ -1,6 +1,7 @@</code>	Hunk: old starts line 1, 6 lines; new starts line 1, 7 lines
leading space	Unchanged context line
<code>-</code>	Line removed from old
<code>+</code>	Line added in new

Diff is how code review works. Push/pull move commits; diff **explains** them.

3. Syntax and examples

`labs/04_git/example.diff`:

```

--- a/hostname.py
+++ b/hostname.py
@@ -1,6 +1,7 @@
import requests

-url = "https://router/restconf/data/Cisco-IOS-XE-native:native/hostname"
+HOST = "https://router"
+url = f"{HOST}/restconf/data/Cisco-IOS-XE-native:native/hostname"
response = requests.get(url, auth=("admin", "admin"), verify=False)
print(response.json())

```

Interpretation: one long URL line was replaced by a `HOST` constant and an f-string. `requests.get` and `print` are unchanged (space prefix). Line count went from 6 to 7 in that hunk (`+1,7`).

Commands:

```

git diff
git diff --stat
git diff HEAD~1

```

`--stat` summarizes files and insert/delete counts without every line.

4. Exam-style understanding

You should be able to:

1. Point to a `-` line and say it was removed.
2. Point to a `+` line and say it was added.
3. Explain `@@` hunk headers at a basic level.
4. Choose `git diff` vs `git diff --staged` based on whether add already happened.
5. Use diff output to describe a change in plain language (as in the hostname URL refactor).

Original practice: After `git add`, `git diff` is empty but `git diff --staged` is not.

5. Hands-on exercise

1. Read `labs/04_git/example.diff` line by line and write a one-sentence summary.
 2. Change a tracked file, run `git diff`, then `git add`, then `git diff --staged`.
 3. After a commit, `git diff HEAD~1`.
 4. Open <https://git-scm.com/docs/git-diff> and skim the unified format section.
-

2.0 Understanding and Using APIs — 20%

Understanding and Using APIs is **one of the two heaviest domains** on CCNA Automation (200-901 CCNAAUTO v1.1). At **20%** of the exam, it outweighs Software Development and Design (15%). If Domain 1 taught you JSON and functions, Domain 2 is where those skills become **HTTP conversations with a platform**.

An API (Application Programming Interface) is a contract: you send a request that follows documented rules; the server returns a structured response. For this exam the dominant style is **REST over HTTP** with **JSON** bodies. You will **construct** requests from documentation, **interpret** status codes, headers, and bodies, **troubleshoot** failures, **utilize** authentication, **compare** API styles, and **construct** Python using the `requests` library.

Study this domain deeper than a glossary. Practice against free endpoints: <https://httpbin.org> (echoes your request), <https://developer.cisco.com/> (real product docs and sandboxes), <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status> (status codes), and <https://requests.readthedocs.io/> (Python client). Lab files: `labs/03_rest_api/rest_client.py` and `labs/03_rest_api/troubleshoot_http.py`.

The exam is 120 minutes. API items are often “here is a doc snippet and an HTTP exchange — what is wrong?” Speed comes from pattern recognition: 401 vs 403, `json=` vs a string body, missing `Accept` header, rate-limit `Retry-After`.

2.1 Construct a REST API request to accomplish a task given API documentation

1. What Cisco expects me to know

Construct means you are given **API documentation** (method, path, headers, query parameters, body schema, auth) and you must **build the request** that performs the task: correct verb, URI, headers, and body. You are not inventing an unofficial URL.

REST here means: **resources** identified by **URIs**, manipulated with **HTTP methods**, typically **stateless**, often **JSON**.

2. Detailed explanation

A REST request has:

1. **Method** — GET (read), POST (create or controller-style action), PUT (replace), PATCH (partial update), DELETE (remove).
2. **URI** — scheme, host, path, query string. Path identifies the resource (`/orgs/{id}/devices`). Query (`?perPage=10`) is not the resource identity; it filters, paginates, or selects.

3. **Headers** — `Accept`, `Content-Type`, `Authorization`, product-specific keys.

4. **Body** — JSON object for POST/PUT/PATCH; GET/DELETE usually have no body.

Stateless: each request carries its own auth and parameters. The server does not need your previous GET to understand this POST (cookies/sessions exist in the web, but REST APIs you study behave as token-per-request).

Documentation-driven construction. Docs typically show:

```
GET https://api.meraki.com/api/v1/organizations/{organizationId}/devices
Header: X-Cisco-Meraki-API-Key: <key>
Header: Accept: application/json
```

Your job: substitute `{organizationId}`, add the key, use GET, do not invent `/devices/list`. If the doc says POST with a JSON body `{"name": "HQ"}`, sending query parameters instead is wrong.

RESTCONF is REST applied to YANG data. Docs specify a path like `/restconf/data/Cisco-IOS-XE-native:native/hostname` and `Accept: application/yang-data+json`. Construction still means: copy the documented path and headers, then fill host and auth.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph LR
    D[API documentation] --> R[Method + URI]
    D --> H[Headers]
    D --> B[Body schema]
    R --> HTTP[HTTP request]
    H --> HTTP
    B --> HTTP
    HTTP --> S[Server]
```

Idempotency (construction implication). GET, PUT, DELETE are typically idempotent: repeating them does not create extra resources. POST often creates a new object each time. If the task is “ensure hostname is edge-01”, PUT/PATCH (or a documented “update” POST) is the right construction; firing POST create twice may duplicate.

3. Syntax and examples

Anatomy

```
GET /get?device=csr1kv&vrf=mgmt HTTP/1.1
Host: httpbin.org
Accept: application/json
```

- Method `GET`

- Path `/get`
- Query `device=csr1kv` and `vrf=mgmt`
- Header `Accept` asks for JSON

From docs to `requests` (lab `get_with_query`)

Docs: “GET `/get` accepts arbitrary query parameters and returns them in `args`.”

```
import requests
r = requests.get(
    "https://httpbin.org/get",
    params={"device": "csr1kv", "vrf": "mgmt"},
    timeout=15,
)
```

`params=` builds the query string and encodes spaces. Do not concatenate `?device=` by hand unless you know encoding rules.

POST JSON (lab `post_json`)

Docs: “POST `/post` accepts a JSON body.”

```
r = requests.post(
    "https://httpbin.org/post",
    json={"hostname": "edge-01", "role": "wan"},
    timeout=15,
)
```

`json=` sets `Content-Type: application/json` and serializes the dict. Using `data="{\"hostname\": ... }"` without the header is a construction error (often 415 later).

Cisco-shaped example (pattern only; host from sandbox docs)

Documentation says:

- Method: GET
- URL: `https://{host}/restconf/data/Cisco-IOS-XE-native:native/hostname`
- Headers: `Accept: application/yang-data+json`
- Auth: HTTP Basic

Construction:

```
url = f"https://{host}/restconf/data/Cisco-IOS-XE-native:native/hostname"
headers = {"Accept": "application/yang-data+json"}
r = requests.get(url, headers=headers, auth=(user, password), verify=False, timeout=15)
```


`verify=False` is a **lab** concession for self-signed certs, not a production default. Docs that require TLS still want HTTPS.

Choosing the method from the task wording

Task in the prompt	Typical method
Retrieve the list of devices	GET
Create a Webex room	POST
Replace the entire hostname resource	PUT
Change one field on a device	PATCH
Remove a webhook subscription	DELETE

Always override this table if the vendor doc says otherwise (some “create” operations are PUT to a known URL).

4. Exam-style understanding

You should be able to:

1. Read a doc table and write the method + path + required headers.
2. Substitute path parameters `{id}` from the scenario.
3. Put filters in the **query string** when docs say query params.
4. Put create/update fields in the **JSON body** when docs show a schema.
5. Avoid mixing: a GET with a JSON body “because JSON is REST” is usually wrong.

Original practice scenario. Docs:

```
POST /networks/{networkId}/vlans
Body JSON: { "id": 30, "name": "GUEST" }
Header: X-Cisco-Meraki-API-Key
```

Task: create VLAN 30 named GUEST in network `N_123`. Constructed request: POST to `/networks/N_123/vlans`, header with API key, body `{"id": 30, "name": "GUEST"}`. Wrong constructions: GET; `/vlans/30` if docs say POST to collection; form-encoded body.

5. Hands-on exercise

Free: <https://httpbin.org> and Postman or `labs/03_rest_api/rest_client.py`.

```
cd labs\03_rest_api
python rest_client.py
```

Then construct, without looking at the file:

1. GET `https://httpbin.org/get` with query `device=edge-01`.
2. POST `https://httpbin.org/post` with JSON `{"hostname": "edge-01"}`.
3. Open <https://developer.cisco.com/>, pick **Meraki API** or **Catalyst Center** getting started, and write (on paper) one GET exactly as the docs specify — method, path, headers — even if you do not send it yet.

Postman: create request, set method, URL, Params tab vs Body raw JSON, Headers. Save the request. You constructed it.

2.2 Describe common usage patterns related to webhooks

1. What Cisco expects me to know

Describe webhooks conceptually: they are **HTTP callbacks**. You **subscribe** a URL; the provider **POSTs** (usually) when an event happens. Patterns: subscription lifecycle, payload shape, **challenge/validation**, retries, security. Connect this to **Observer** (1.6).

2. Detailed explanation

Polling is the opposite pattern: your script GETs `/events` every 30 seconds. **Webhooks** invert it: the platform is the HTTP **client** for a moment, and **your** automation is the HTTP **server**.

Common usage patterns

1. **Subscribe**. You call the vendor's REST API (this is still 2.1) to register `targetUrl`, event types (message created, device down), and maybe a secret. Example conceptual resource: `POST /webhooks` with `{"url": "https://nso.example/hook", "events": ["alarms"]}`.
2. **Receive**. Your endpoint must be reachable (public HTTPS, or a tunnel for labs). It must answer quickly (2xx) so the provider does not mark you unhealthy.
3. **Challenge / validation (handshake)**. Many providers prove you control the URL: they send a GET or POST with a `challenge` string; you must echo it back. If you fail, the subscription is not activated. This stops people from pointing webhooks at victim servers (DDoS amplifier / unsolicited traffic).
4. **Event delivery**. Later, JSON body describes the event. You parse (1.2) and act (create a ticket, send Webex, shut an interface — carefully).
5. **Retry**. If your endpoint returns 5xx or times out, platforms typically **retry** with backoff. Your handler should be **idempotent** where possible: processing the same event twice should not create two change tickets.
6. **Unsubscribe / rotate**. Delete the webhook resource when the receiver dies, or you will generate failures and lockouts. Rotate secrets.

7. **Security patterns.** HMAC signatures in a header, shared secrets, allow-list source IPs, TLS. You authenticate **the sender** (is this really Meraki/Webex?) not only your users.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant You as You automation
    participant API as Platform REST API
    participant Hook as Your webhook URL
    You->>API: POST /webhooks subscribe
    API->>Hook: Challenge / validation
    Hook->>API: Echo challenge 200
    Note over API: Event occurs
    API->>Hook: POST event JSON
    Hook->>API: 200 OK
    Hook->>Hook: Act on event
```

Where Cisco uses this idea. Webex webhooks for messages/memberships; dashboard platforms notifying on network events; ChatOps bots. You do not need every product's field list; you need the **pattern**.

Webhook vs websocket vs polling

Pattern	Who initiates after setup	Typical exam association
Polling	You, on a timer	Simple scripts, rate-limit heavy
Webhook	Platform, per event	Observer, callbacks
WebSocket	Persistent connection	Streaming (know it exists; webhook is the named topic)

3. Syntax and examples

There is no single webhook standard. Teaching syntax is **subscribe with REST, receive POST JSON**.

Subscribe (you are the REST client — 2.1 / 2.9)

```
import requests

r = requests.post(
    "https://httpbin.org/post",
    json={
        "name": "ccnaauto-lab",
        "targetUrl": "https://example.com/hooks/alerts",
        "resource": "alarms",
        "event": "created",
    },
    headers={"Authorization": "Bearer TOKEN"},
    timeout=15,
)
```

httpbin will echo the JSON; a real API would return a webhook `id` for later DELETE.

Challenge response (you are the HTTP server)

Pseudo-handler: if the body has `"challenge"`, return that string as the response body with `200` and `Content-Type: text/plain` (or whatever the vendor documents). Only after that will event POSTs arrive.

Event POST you might receive

```
POST /hooks/alerts HTTP/1.1
Host: example.com
Content-Type: application/json
X-Signature: sha256= ...

{"event": "device_down", "serial": "Q2XX-1111", "ts": "2026-08-16T15:00:00Z"}
```

Your code: verify signature, `json.loads` body, branch on `event`, return `204` or `200` quickly, do slow work asynchronously if needed.

Retry pattern. First POST → your server 503. Provider waits, POSTs again with the same event id. If you created a ticket on the first partial attempt, check event id before creating another.

4. Exam-style understanding

You should be able to:

1. Define webhook as an HTTP callback on an event (push), not a poll.
2. Describe subscribe → (optional challenge) → event POST → 2xx → retry on failure.
3. Relate to Observer: platform is subject; your URL is observer.
4. Explain why challenge exists: prove control of the URL.
5. Explain why 2xx matters: suppress retries / mark healthy.

Original practice scenario. A Webex bot never fires. Checklist: subscription created? Public HTTPS URL? Challenge handler implemented? Returning 401 on events because you reused user-auth instead of signature check? Returning 500 and the platform gave up after N retries?

5. Hands-on exercise

Free tools: httpbin + a local receiver.

1. POST a fake subscription body to <https://httpbin.org/post> using `rest_client.py` style `json=`.
2. Use Python stdlib to listen locally:

```

from http.server import BaseHTTPRequestHandler, HTTPServer
import json

class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        length = int(self.headers.get("Content-Length", "0"))
        body = json.loads(self.rfile.read(length) or b"{}")
        print("event", body)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"ok")

HTTPServer(("127.0.0.1", 8765), Hook).serve_forever()

```

3. In another terminal: `python -c "import requests; requests.post('http://127.0.0.1:8765', json={'event': 'up'})"`.
4. Write the sequence diagram in your notes. Optional: ngrok or Cloudflare Tunnel if you later test a real Webex webhook (free tier; not required today).

2.3 Describe the constraints when consuming APIs

1. What Cisco expects me to know

Describe limits that exist **when you are the client**: rate limits, pagination, payload size, versioning, TLS, timeouts, idempotency, required headers, auth scopes. Consuming an API is not “unlimited GET in a `while True`.”

2. Detailed explanation

Vendors protect shared control planes. Your script must live inside those constraints or you will see **429**, truncated lists, `413`, TLS errors, or silent misses (page 2 never fetched).

Rate limits. A quota such as “N requests per second per token.” Exceeding it yields **429 Too Many Requests**, often with `Retry-After`. Constraint: backoff, cache GETs, use webhooks instead of polling.

Pagination. Large collections are not one JSON array of 50,000 devices. APIs return a page plus a `next` link, cursor, or `page / offset` query params. Constraint: loop until there is no next page, or you only automated the first 100 devices.

Payload size. POST bodies and responses have limits. Sending a 20 MB config blob may fail. Responses may truncate. Constraint: chunk, compress if documented, or use a bulk API.

Versioning. Paths include `/api/v1/` or headers `Accept: application/vnd.example.v2+json`. Constraint: pin the version in your client; do not assume v1 lives forever. Breaking changes happen on a new version.

TLS / certificates. Production APIs require HTTPS. Self-signed sandbox certs fail verification (`verify=True` default in `requests`). Constraint: install the CA, or lab-only `verify=False` . Corporate proxies intercept TLS.

Timeouts and retries. Networks fail. `requests` without `timeout=` can hang the exam lab **and** a production controller. Constraint: set timeouts; retry **idempotent** GETs; be careful retrying POST.

Idempotency. Constraint: design writes so a retried PUT is safe. Some APIs support `Idempotency-Key` headers.

Auth and scope. A valid token might still be forbidden for DELETE (403). Constraint: least privilege; different keys for read-only inventory vs change.

Schema and content types. Constraint: send `Content-Type: application/json` (or `yang-data+json`). Wrong type → **415**. Extra unknown fields may be ignored or **400**.

Filtering vs client-side loops. Docs may allow `?name=edge-01` . Constraint: use server-side filters to stay under rate limits.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
graph TD
    C[Your client] --> L{Constraints}
    L --> R[Rate limit / 429]
    L --> P[Pagination]
    L --> S[Payload size]
    L --> V[Versioned path]
    L --> T[TLS]
    L --> I[Idempotent retries]
```

3. Syntax and examples

Timeout (always in this course)

```
r = requests.get(url, timeout=15)
# or (connect timeout, read timeout)
r = requests.get(url, timeout=(5, 30))
```

Pagination loop (pattern)

```
url = "https://api.example.com/v1/devices"
while url:
    r = requests.get(url, headers=headers, timeout=15)
    r.raise_for_status()
    data = r.json()
    for device in data["items"]:
        print(device["name"])
    url = data.get("next") # None when done
```

If you only `GET` once, you silently skipped remaining items — a **constraint bug**, not an HTTP error.

Rate limit handling

```
import time

r = requests.get(url, headers=headers, timeout=15)
if r.status_code == 429:
    wait = int(r.headers.get("Retry-After", "5"))
    time.sleep(wait)
```

Version in the URI

```
https://api.meraki.com/api/v1/organizations
```

`v1` is a constraint: your code is coupled to that contract.

TLS

```
requests.get(url, timeout=15) # verify=True
requests.get(url, timeout=15, verify=False) # lab only
```

4. Exam-style understanding

You should be able to:

1. Name several constraints: rate limits, pagination, payload size, versioning, TLS, timeouts, idempotency.
2. Map 429 → rate limit; incomplete inventory → missed pagination; 415 → content type.
3. Explain why a tight poll loop is a bad consumer.
4. Explain why `timeout` is part of consuming APIs correctly.
5. Distinguish “API is down” (5xx) from “you ignored the rules” (4xx / missing pages).

Original practice scenario. Script lists 100 access points; the site has 350. Status 200. Root cause: `perPage=100` and no follow of `next`. Not a Cisco bug — a consumer constraint miss.

5. Hands-on exercise

1. Call `GET https://httpbin.org/delay/3` with `timeout=1` and with `timeout=15`. Observe timeout vs success.
2. Call `GET https://httpbin.org/status/429` and feed it through `labs/03_rest_api/troubleshoot_http.py` logic.
3. Read any Cisco API “rate limit” or “pagination” paragraph on <https://developer.cisco.com/> (Meraki and Catalyst Center both document this). Copy the constraint into your notes in your own words.

4. Write a three-line checklist you will apply to every new API: version, auth header, page loop, timeout, backoff.

2.4 Explain common HTTP response codes associated with REST APIs

1. What Cisco expects me to know

Explain the codes that show up with REST APIs — what they **mean**, not only the number. Focus on: **200, 201, 204, 400, 401, 403, 404, 409, 415, 429, 500, 503**. Official overview: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

2. Detailed explanation

HTTP status codes are three-digit integers in the **status line**. Classes:

Range	Class	Client takeaway
1xx	Informational	Rare in REST JSON APIs
2xx	Success	Request understood and accepted
3xx	Redirect	Follow <code>Location</code> or fix the client
4xx	Client error	Your request/auth/path/body
5xx	Server error	Platform or upstream; retry may help

Success codes you must separate

Code	Name	Typical REST meaning
200	OK	GET succeeded; PUT/PATCH/POST sometimes return 200 with a body
201	Created	POST created a resource; often <code>Location</code> header with the new URI
204	No Content	Success, empty body (DELETE or PUT with nothing to say)

If you `r.json()` on a **204**, parsing fails — the code was still success.

Client errors you must separate

Code	Name	Typical REST meaning
400	Bad Request	Malformed JSON, missing field, invalid VLAN ID
401	Unauthorized	Authentication failed: no/bad credentials
403	Forbidden	Authenticated, but not allowed (RBAC, license, scope)
404	Not Found	Bad path, wrong id, resource deleted, or hidden (some APIs 404 instead of 403)
409	Conflict	State clash: duplicate name, VLAN exists, edit conflict
415	Unsupported Media Type	Content-Type wrong (sent XML or form, expected JSON)
429	Too Many Requests	Rate limit (2.3)

401 vs 403 is the highest-value distinction. 401: the server does not accept who you claim to be (or you claimed nobody). 403: it knows who you are and still says no.

Server errors

Code	Name	Typical REST meaning
500	Internal Server Error	Unhandled failure on the server
503	Service Unavailable	Overloaded, maintenance, retry later

5xx does **not** prove your JSON is wrong. Check 4xx first when the body schema is suspicious, but do not “fix” a payload in response to 503.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TD
    S[Status code] --> A{Class}
    A -->|2xx| OK[Parse body unless 204]
    A -->|401| AUTH[Fix credentials / token / key]
    A -->|403| RBAC[Fix permissions]
    A -->|404| PATH[Fix URI / id]
    A -->|400 or 415| BODY[Fix JSON / Content-Type]
    A -->|409| STATE[Resource already exists or clash]
    A -->|429| WAIT[Backoff Retry-After]
    A -->|5xx| SRV[Retry later; check status page]

```

3. Syntax and examples

Status line:

```
HTTP/1.1 201 Created
Location: /networks/N_123/vlans/30
Content-Type: application/json
```

httpbin can mint codes (lab `status_codes`):

```
r = requests.get("https://httpbin.org/status/404", timeout=15)
print(r.status_code, r.reason) # 404 NOT FOUND
```

Meaning attached to a constructed request

You sent	Likely code if wrong
GET with typo in path	404
POST valid JSON, missing auth header	401
POST with intern's read-only key	403
POST <code>{id: 30}</code> without quotes around <code>id</code> — invalid JSON	400
POST XML to a JSON-only API	415
POST same VLAN twice	409 (if vendor uses conflict)
Loop GET 1000/min	429
Sandbox down	500 or 503

201 vs 200 on create. Some APIs return 200 for POST. Docs win. The **concept** of 201 is “created.”

4. Exam-style understanding

You should be able to:

1. Explain each listed code in one sentence.
2. Contrast 200 / 201 / 204.
3. Contrast 401 / 403 / 404.
4. Contrast 400 / 409 / 415 / 429.
5. Contrast 500 / 503 vs 4xx (who should change what).

Original practice scenario. Response `401` with body `{"error": "invalid token"}` — do not debug the YANG path. Response `404` with a correct token — debug the URI. Response `204` after DELETE — success; do not report “empty body means failure.”

5. Hands-on exercise

```
cd labs\03_rest_api
python rest_client.py
```

Watch the “Status code lab” section print 200, 201, 400, 401, 403, 404, 429, 500. Then:

```
python troubleshoot_http.py
```

Write a one-line “what I would change” for each of 401, 404, 429, 200. Bookmark

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

2.5 Troubleshoot a problem given the HTTP response code, request and API documentation

1. What Cisco expects me to know

Troubleshoot = identify **what is wrong** and a **solution**, given three artifacts: the **request**, the **response code** (and usually headers/body), and **API documentation**. This is Domain 2’s applied skill. 2.4 is the codebook; 2.5 is the diagnosis.

2. Detailed explanation

Work a **fixed order** so you do not guess:

1. **Compare the request to the docs** — method, path (including prefix `/api/v1` or `/restconf/data`), path parameters, query vs body, required headers, auth scheme.
2. **Classify the status code** (2.4).
3. **Read the body** — vendors often return `{"message": "Unknown network"}` which beats guessing.
4. **Read headers** — `WWW-Authenticate`, `Retry-After`, `Content-Type`, `Location`.
5. **Propose one change** — the smallest fix that matches both the code and the docs.

Common mismatch patterns

Evidence	Likely fault	Solution
401, docs say API key header, request used Basic	Wrong auth mechanism	Send the documented header/token
403, 401 already ruled out	Wrong RBAC / scope / org	Use a key with permission; correct org id
404, host is correct	Path, id, or API version	Copy path from docs; verify the resource exists with GET list
400, JSON looks “fine”	Field name, type, or enum	Match schema ("enabled": true not "Enabled": "yes")
415	Content-Type	application/json or application/yang-data+json
409	Duplicate or bad state	GET first; use PATCH; pick another name
429	Too fast	Honor Retry-After ; slow the loop
500/503, request matches docs	Platform	Retry; different sandbox; do not rewrite JSON blindly
200 but empty list	Pagination or filter	Follow next ; drop bad query param
TLS error, never a status code	Certificate	Lab verify=False or trust the CA

Do not stop at the number. 404 on GET /device vs documented GET /devices is a path typo. 404 on GET /devices/Q2XX with a valid key may be a **wrong serial** — still a client problem.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TD
    A[Request + code + docs] --> B{Code class}
    B -->|401/403| C[AuthN vs AuthZ vs header name]
    B -->|404| D[Path / id / version vs docs]
    B -->|400/415/409| E[Body and Content-Type vs schema]
    B -->|429| F[Rate limit]
    B -->|5xx| G[Server; retry]
    B -->|2xx but wrong data| H[Pagination / parse / wrong resource]

```

3. Syntax and examples

Lab diagnostic helper: labs/03_rest_api/troubleshoot_http.py .

```
def diagnose(response, api_doc_expected: str) → str:
    code = response.status_code
    if code == 401:
        return "Authentication failed. Check username/password, token, or API key."
    if code == 403:
        return "Authenticated but not authorized. The credential lacks permission."
    if code == 404:
        return f"Resource not found. Confirm the URL path. Docs expected: {api_doc_expected}"
    if code == 400:
        return "Bad request. Inspect JSON body/query params against the API schema."
    if code == 415:
        return "Unsupported media type. Set Content-Type to application/json (or yang-data+json)."
```

```
    if code == 429:
        retry = response.headers.get("Retry-After", "unknown")
        return f"Rate limited. Wait Retry-After={retry} seconds, then retry."
    if 500 ≤ code ≤ 599:
        return "Server-side failure. Retry later; do not assume your payload is wrong."
    if 200 ≤ code ≤ 299:
        return "Success. Parse the body."
    return f"Unexpected status {code}."
```

This is a **starting** tree. Real troubleshooting still diffs the request against docs.

Worked original example 1

- **Docs:** GET /api/v1/organizations/{orgId}/devices header X-Cisco-Meraki-API-Key .
- **Request:** GET /api/v1/organizations/devices with that header (org id missing — path collapsed).
- **Code:** 404.
- **Wrong:** “API is down.”
- **Right:** path missing {orgId} . **Solution:** insert the organization id segment.

Worked original example 2

- **Docs:** POST JSON, Content-Type: application/json .
- **Request:** POST with body hostname=edge-01 and Content-Type: application/x-www-form-urlencoded .
- **Code:** 415 or 400.
- **Solution:** send a JSON object with json= in requests or raw JSON in Postman.

Worked original example 3

- **Docs:** Basic auth, RESTCONF, Accept: application/yang-data+json .
- **Request:** GET correct URL, Basic auth, Accept: application/json .
- **Code:** 406 Not Acceptable (if implemented) or 400/415-family. Some devices still answer. If 404, RESTCONF might be disabled — that is also a “request vs platform capability” issue.
- **Solution:** use the documented Accept value; confirm feature enablement if the path never exists.

Worked original example 4

- **Request:** matches docs.
- **Code:** 429, header `Retry-After: 8`.
- **Solution:** wait 8 seconds (and slow the client). Not a new API key.

4. Exam-style understanding

You should be able to:

1. Produce a **fault + fix** pair, not only a code definition.
2. Use docs as the source of truth for path and headers.
3. Prefer body error messages when present.
4. Avoid “500 means my VLAN is wrong.”
5. Handle the 2xx-but-wrong-result case (pagination, parsing `text` instead of JSON).

Original practice: Given `401` and a request that used `Authorization: Bearer` while docs show `X-Cisco-Meraki-API-Key`, the fix is the header, not the URL.

5. Hands-on exercise

```
cd labs\03_rest_api
python troubleshoot_http.py
```

Then, using Postman or Python, **manufacture** faults against httpbin and write the diagnosis:

Request	Expected code	Your written fix
GET <code>/status/401</code>	401	Supply valid basic/token/key
GET <code>/status/404</code>	404	Correct the documented path
POST <code>/post</code> with <code>data="not-json"</code> and JSON Content-Type	200 from httpbin (it accepts)	Switch to a strict API or inspect that httpbin is a poor 415 demo
GET <code>/status/415</code>	415	Set Content-Type / Accept per docs
GET <code>/status/429</code>	429	Backoff

For a stricter 415 demo, RESTCONF on a sandbox with the wrong `Content-Type` is ideal when you reach Domain 3/5. Until then, reason from docs + codes using the tables above.

Craft one **wrong path** GET to <https://httpbin.org/status/404> and one **wrong auth** GET to `/basic-auth/admin/s3cret` **without** `auth=`. Confirm 401. Fix with `auth=("admin", "s3cret")` as in `rest_client.py`.

2.6 Interpret the parts of an HTTP response (response code, headers, body)

1. What Cisco expects me to know

Interpret means **read provided output** and extract meaning from the three parts: **status code**, **headers**, **body**. You may be shown a raw HTTP dump or `requests` prints. You do not construct here; you read.

2. Detailed explanation

An HTTP response is:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 348
Retry-After: 0
Server: gunicorn

{"args": {"device": "csr1kv"}, "url": "https://httpbin.org/get?device=csr1kv"}
```

1. Response code (status line). `HTTP/1.1` is the version. `200` is the code. `OK` is the reason phrase (informational; the number is authoritative). Interpretation starts here (2.4).

2. Headers. Name/value metadata. Case-insensitive names. Useful headers:

Header	What you interpret
<code>Content-Type</code>	How to parse the body (<code>application/json</code> , <code>application/yang-data+json</code> , <code>text/plain</code>)
<code>Location</code>	URI of a created resource (201)
<code>Retry-After</code>	Seconds (or HTTP date) to wait (429/503)
<code>WWW-Authenticate</code>	Auth scheme the server expects (Basic realm=...)
<code>Set-Cookie</code>	Session (less common in token REST)
<code>Allow</code>	Methods on this resource (after 405)
<code>X-Request-Id</code> / <code>Tracking-Id</code>	Correlation for TAC / vendor support
<code>Content-Length</code>	Body size; <code>0</code> with 204 is normal

Headers are **not** the JSON fields. `enabled: true` in a header would be exotic; it belongs in the body.

3. Body. Bytes after the blank line that ends headers. May be JSON, XML, HTML error page, or empty. Interpretation:

- If `Content-Type` is JSON, parse to dict/list (1.2, `response.json()`).
- Error bodies often contain `message` , `error` , `ietf-restconf:errors` .
- Empty body + 204 = success.
- HTML body + 200 on an API URL often means you hit a **web UI** or proxy login, not the API — interpret as “wrong URL or auth portal.”

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

flowchart TB
    R[HTTP response] --> C[Status code]
    R --> H[Headers]
    R --> B[Body]
    C --> I[Success vs class of failure]
    H --> I
    B --> I

```

requests mapping

HTTP part	<code>requests</code> attribute
Status code	<code>response.status_code</code>
Reason	<code>response.reason</code>
Headers	<code>response.headers</code> (case-insensitive dict)
Body text	<code>response.text</code>
Body bytes	<code>response.content</code>
Parsed JSON	<code>response.json()</code>

Lab `show_response` in `rest_client.py` prints URL, status, Content-Type, then JSON.

3. Syntax and examples

Interpret this original dump

```

HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm="Restricted"
Content-Type: application/json

{"authenticated": false}

```

- Code: authentication failed.
- Header: server wants **Basic** auth, not an API key.

- Body: confirms not authenticated. **Action:** add `Authorization: Basic ...` (2.7).

Interpret this original dump

```
HTTP/1.1 201 Created
Location: /api/v1/rooms/abc123
Content-Type: application/json

{"id": "abc123", "title": "CCNAAUTO"}
```

- Code: resource created.
- `Location`: GET this path to fetch the room.
- Body: JSON with the new `id`. **Action:** store `id` for later membership APIs.

Interpret this original dump

```
HTTP/1.1 204 No Content
Content-Length: 0
```

- Success, no body. Do not call `.json()`.

Interpret requests output

```
Status: 200 OK
Content-Type: application/json
{'args': {'device': 'csr1kv', 'vrf': 'mgmt'}, ...}
```

Query params were accepted and echoed. Construction of GET+params worked.

Trap: pretty-printed Python dict vs JSON. If the exam shows `True / None`, you are looking at **Python**. If it shows `true / null`, you are looking at **JSON** (1.1).

4. Exam-style understanding

You should be able to:

1. Point to code, headers, and body in a raw response.
2. Say what `Content-Type` implies for parsing.
3. Use `Location` and `Retry-After` correctly.
4. Know `204` $\Rightarrow$ empty body is OK.
5. Distinguish `response.text` (string) from `response.json()` (structure).

Original practice: A dump with 200 and HTML `<title>Login</title>` — interpret as “not the JSON API”; check base URL and auth. A dump with 200 and JSON `[]` — success with empty collection, not necessarily 404.

5. Hands-on exercise

Run `python labs/03_rest_api/rest_client.py` and for **each** printed block label:

1. The numeric code and class (2xx/4xx).
2. One header you would use in troubleshooting.
3. Whether the body is JSON and one field you would read in a script.

In Postman, send `GET https://httpbin.org/get` and click **Status**, **Headers**, **Body** panes separately. Say one sentence per pane.

Optional: `curl -i https://httpbin.org/get` (`-i` includes headers). Interpret the raw text.

2.7 Utilize common API authentication mechanisms: basic, custom token, and API keys

1. What Cisco expects me to know

Utilize three mechanisms: **HTTP Basic**, **custom token** (often Bearer), and **API keys**. Know **how they appear on the wire** (which header, which encoding) and how to send them with `requests`. Never commit secrets; use `labs/.env`.

2. Detailed explanation

APIs must know **who** the caller is (authentication). Authorization (what you may do) is separate (403 vs 401).

HTTP Basic. The client sends username and password on every request, combined as `username:password`, encoded with **Base64** (not encryption — anyone who sees the header can decode it). Hence **TLS is mandatory**.

```
Authorization: Basic YWRtaW46czNjcmV0
```

`YWRtaW46czNjcmV0` is Base64 for `admin:s3cret`. The server decodes and checks. RESTCONF on IOS XE often uses Basic in labs. `requests auth=(user, pass)` builds this header for you.

API keys. A long secret string issued by the platform (Meraki dashboard, some SaaS). Common pattern: a **custom header**:

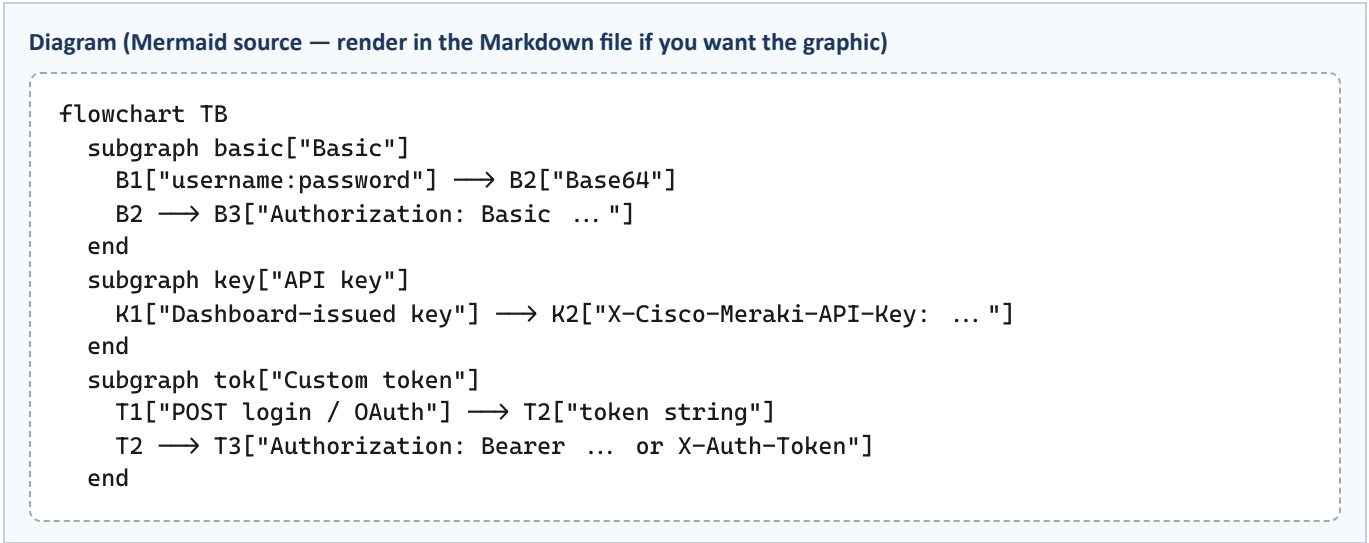
```
X-Cisco-Meraki-API-Key: 1234567890abcdef
```

Sometimes a query parameter `?apiKey=` (worse: keys land in logs). Sometimes `Authorization: Bearer <key>`. **Utilize** means: put the key **where the docs say**, not where another product puts it.

Custom tokens. After you authenticate (POST username/password to `/login` or an OAuth endpoint), the server returns a **token** (opaque string or JWT). Subsequent requests send:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 ...
```

Catalyst Center (DNA Center) classic flow: POST to `/dna/system/api/v1/auth/token` with Basic, receive a token, then send `X-Auth-Token: <token>` — that is a **custom token header**, not the Meraki key header. “Custom token” on the blueprint covers Bearer **and** vendor-specific token headers.



Comparison

Mechanism	What you send	Typical Cisco-shaped use	Risk
Basic	Base64 user:pass every request	RESTCONF lab devices	Password in header; must be HTTPS
API key	Static secret in a header	Meraki	Key = password; rotate; never Git
Custom token	Short-lived string after login	Catalyst Center, Webex, OAuth	Expire/refresh; still HTTPS

Webex: you create an integration or bot and get a token used as `Authorization: Bearer` . That is custom/Bearer token, not Basic.

3. Syntax and examples

Lab functions in `labs/03_rest_api/rest_client.py` .

Basic with `requests`

```
r = requests.get(
    "https://httpbin.org/basic-auth/admin/s3cret",
    auth=("admin", "s3cret"),
    timeout=15,
)
```

Manual equivalent (you should understand, rarely write):

```
import base64
token = base64.b64encode(b"admin:s3cret").decode("ascii")
headers = {"Authorization": f"Basic {token}"}
```

API key header (Meraki pattern, echoed by httpbin /headers)

```
headers = {
    "X-Cisco-Meraki-API-Key": "fake-key-for-lab",
    "Accept": "application/json",
}
r = requests.get("https://httpbin.org/headers", headers=headers, timeout=15)
```

httpbin returns the headers it received so you can **see** the key name. On a real Meraki call the URL would be `https://api.meraki.com/api/v1/ ...`

Bearer / custom token

```
headers = {"Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.lab"}
r = requests.get("https://httpbin.org/bearer", headers=headers, timeout=15)
```

Catalyst Center **pattern** (host from sandbox docs):

```
auth_url = f"https://{host}/dna/system/api/v1/auth/token"
r = requests.post(auth_url, auth=(user, password), verify=False, timeout=15)
token = r.json()["Token"]
headers = {"X-Auth-Token": token, "Content-Type": "application/json"}
requests.get(f"https://{host}/dna/intent/api/v1/network-device", headers=headers,
verify=False, timeout=15)
```

Two mechanisms in sequence: Basic to **obtain** a custom token, then token header to **utilize** the API.

Wrong mixes (will 401)

- Meraki key placed in `Authorization: Basic`
- Bearer token sent as `X-Cisco-Meraki-API-Key`
- Basic user/pass in the JSON body when docs say the Authorization header

4. Exam-style understanding

You should be able to:

1. Build a Basic header conceptually (Base64 of `user:pass`).
2. Place an API key in a **custom header** when documented (Meraki).
3. Place a token in `Authorization: Bearer` or a vendor token header.
4. Choose the mechanism from docs, not from habit.
5. Explain why Base64 is not confidentiality.

Original practice: Docs: “Authenticate with header `X-Cisco-Meraki-API-Key`.” Request includes `auth=`
(`"key", ""`) Basic. Result 401. Fix: `headers={"X-Cisco-Meraki-API-Key": key}`.

5. Hands-on exercise

```
cd labs\03_rest_api
python rest_client.py
```

Study the Basic, API-key, and Bearer sections of the printout.

1. Decode a Basic blob: in Python `import base64; base64.b64decode("YWRtaW46czNjcmV0")`.
2. Send `/basic-auth/admin/s3cret` **without** auth; confirm 401; add `auth=`.
3. Read Meraki getting started: <https://developer.cisco.com/meraki/api-v1/getting-started/> — note the **exact** header name.
4. Put a fake key in `labs/.env` (not in Git). Load it later with `python-dotenv` when you hit Domain 3 labs.

2.8 Compare common API styles (REST, RPC, synchronous, and asynchronous)

1. What Cisco expects me to know

Compare four ideas that are not all the same axis: **REST** vs **RPC** (style of interface), and **synchronous** vs **asynchronous** (style of waiting). Advantages, disadvantages, use cases.

2. Detailed explanation

REST (Representational State Transfer). You manipulate **resources** (nouns) at URIs with a uniform set of HTTP methods. `GET /interfaces/GigabitEthernet1` retrieves a representation (JSON). `PATCH` updates fields. Benefits: cacheable GETs, uniform interface, maps to HTTP well, easy to document per resource. Drawbacks: awkward for “reboot the box” (not a noun) — vendors add POST actions or RPC-like URLs. Statelessness simplifies scale.

RPC (Remote Procedure Call). You call **functions** (verbs) on a server: `CreateVlan(id=30)` , `GetHostname()` . NETCONF is XML RPC (`<rpc><get-config>`). JSON-RPC and gRPC are RPC families. SOAP is XML RPC over HTTP. Benefits: natural for operations and transactions; strong schemas (YANG, protobuf). Drawbacks: each API invents method names; harder to cache; you must know the procedure catalog. RESTCONF is “REST-like access to the same YANG models NETCONF RPCs use.”

	REST	RPC
Center	Resources / URIs	Procedures / methods
HTTP methods	Uniform GET/POST/PUT/PATCH/DELETE	Often POST everything
Example	<code>GET /vlans/30</code>	<code>POST /rpc body getVlan {id:30}</code>
Cisco-shaped	Meraki, RESTCONF, Webex	NETCONF, some SOAP, gRPC telemetry
Advantage	Predictable verbs, web-friendly	Expressive actions, transactional configs
Disadvantage	Verbs-as-nouns feel forced	Less uniform; client per method

Synchronous. The client sends a request and **waits** for the complete result in that HTTP response. `GET /hostname` → 200 `{"hostname": "edge-01"}` . Simple. Blocks the caller. Bad for “upgrade IOS” that takes 10 minutes — you would timeout (2.3).

Asynchronous. The call **starts** work and returns quickly with a **job id** (202 Accepted is a common HTTP pattern) or you use a webhook/callback (2.2) when done. You poll `GET /jobs/{id}` or wait for a notification. Benefits: long tasks, scale. Drawbacks: more states (queued, running, failed), more code, eventual consistency.

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```
sequenceDiagram
    participant C as Client
    participant A as API
    Note over C,A: Synchronous
    C->>A: POST /vlans
    A->>C: 201 + VLAN JSON
    Note over C,A: Asynchronous
    C->>A: POST /os-upgrade
    A->>C: 202 + jobId
    C->>A: GET /jobs/jobId
    A->>C: 200 running
    A->>C: webhook completed
```

Cross-cutting. REST can be sync (most CRUD) or async (202 + job). RPC can be sync (NETCONF `<rpc-reply>` in the same SSH session) or async (notification streams). Do not say “REST is synchronous and RPC is asynchronous” — those are **different comparisons**.

Use cases

Need	Style
CRUD on inventory objects	REST, synchronous
Replace whole config datastore	NETCONF RPC (edit-config)
Image install	Asynchronous job + poll or webhook
Chat message send	REST POST, usually synchronous 200
Streaming telemetry	Often RPC/gRPC, asynchronous stream

3. Syntax and examples

REST synchronous

```
r = requests.post("https://httpbin.org/post", json={"id": 30, "name": "GUEST"}, timeout=15)
print(r.status_code) # 200 from httpbin; a real API might 201
print(r.json()["json"])
```

You waited; the body is the result.

RPC-shaped HTTP (illustrative)

```
POST /api/rpc HTTP/1.1
Content-Type: application/json

{"method": "setHostname", "params": {"hostname": "edge-01"}, "id": 1}
```

One URL, method **inside** the body. Compare to REST `PUT /hostname` with `{"hostname": "edge-01"}`.

Asynchronous REST pattern

```
POST /api/v1/images/activate HTTP/1.1

HTTP/1.1 202 Accepted
Location: /api/v1/jobs/7f3c
Content-Type: application/json

{"jobId": "7f3c", "status": "accepted"}
```

Follow with GET `/api/v1/jobs/7f3c` until `status` is `success` or `failed`, or register a webhook.

NETCONF as RPC (preview of later domains)

```
<rpc message-id="101">
  <get-config>
    <source><running /></source>
  </get-config>
</rpc>
```

The operation is `get-config`, not `GET /running`. Same YANG data RESTCONF would expose as GET.

4. Exam-style understanding

You should be able to:

1. Contrast REST (resources, uniform verbs) vs RPC (named procedures).
2. Contrast sync (wait for result) vs async (job id / callback).
3. Give an advantage and disadvantage of each.
4. Avoid mixing the two axes.
5. Map NETCONF → RPC, Meraki → REST, OS upgrade → often async, GET hostname → sync.

Original practice: “POST `/reboot` that returns 200 when the box is already back” would be a **synchronous** lie if reboot takes 3 minutes — the API should be **async** 202. “POST `/rpc` with `getInterface`” vs `GET /interfaces/Gi1` is REST vs RPC.

5. Hands-on exercise

1. Table: REST vs RPC vs sync vs async with one Cisco-flavored example each.
2. Using `httpbin`, perform a synchronous POST and interpret 200 + echoed JSON (`rest_client.py`).
3. Simulate async: `GET https://httpbin.org/status/202` and write the next client step (`GET /jobs/{id}` or `webhook`).
4. Open a NETCONF example in `labs/06_yang_netconf_restconf/sample_netconf_get-config.xml` and a RESTCONF JSON in `sample_restconf_get.json`. Say which is RPC-shaped and which is REST-shaped.

2.9 Construct a Python script that calls a REST API using the requests library

1. What Cisco expects me to know

Construct a Python script with `requests`: send GET/POST (and friends), pass `params`, `json=`, `headers`, `auth`, `timeout`, then read `status_code` and `.json()`. Official docs: <https://requests.readthedocs.io/>. The teaching script is `labs/03_rest_api/rest_client.py`.

2. Detailed explanation

`requests` is the de facto HTTP library for this exam. It is **not** in the Python standard library (`pip install requests` , already in `labs/requirements.txt`).

Mental model: you build a Python function that (1) composes URL + headers + body from documentation (2.1), (2) adds auth (2.7), (3) enforces timeout (2.3), (4) inspects `status_code` (2.4–2.6), (5) parses JSON (1.2).

Core call signatures

```
requests.get(url, params=None, headers=None, auth=None, timeout=..., verify=True)
requests.post(url, json=None, data=None, headers=None, auth=None, timeout=...)
```

- `params` : dict → query string.
- `json` : dict/list → JSON body + Content-Type.
- `data` : form fields or raw bytes — **not** the same as `json=` .
- `headers` : dict of extra headers.
- `auth` : tuple (user, password) for Basic.
- `timeout` : seconds (or tuple). **Always set it.**
- `verify` : TLS validation.

Response object — you will use:

Attribute / method	Use
<code>r.status_code</code>	int 200, 401, ...
<code>r.reason</code>	OK , Not Found
<code>r.headers</code>	header map
<code>r.text</code>	decoded body string
<code>r.content</code>	raw bytes
<code>r.json()</code>	parse JSON → dict/list
<code>r.raise_for_status()</code>	throw if 4xx/5xx
<code>r.url</code>	final URL after redirects / params

Construction errors in Python

Bug	What happens
<code>json.dumps(payload)</code> passed to <code>json=</code>	Double-encoded string in JSON
<code>data=payload_dict</code> without headers	Form encoding, likely 415 on JSON APIs
No <code>timeout</code>	Can hang forever
<code>r.json()</code> on 204 or HTML	<code>ValueError</code>
Ignoring <code>status_code</code>	Treat HTML error pages as success

Diagram (Mermaid source — render in the Markdown file if you want the graphic)

```

graph LR
    A["requests.get/post"] --> B[Response]
    B --> C["status_code"]
    B --> D["headers"]
    B --> E["json() or text"]

```

3. Syntax and examples

Install/import:

```
import requests
```

GET with query parameters

```

r = requests.get(
    "https://httpbin.org/get",
    params={"device": "csr1kv", "vrf": "mgmt"},
    timeout=15,
)
print(r.status_code)
print(r.json()["args"])

```

`params` produces `?device=csr1kv&vrf=mgmt`. `r.json()` is a dict; `["args"]` is httpbin's echo of the query.

POST JSON body

```

payload = {"hostname": "edge-01", "role": "wan"}
r = requests.post("https://httpbin.org/post", json=payload, timeout=15)
print(r.json()["json"]) # echoed body as object

```

`json=payload` is the correct construction. Contrast:

```
# usually wrong for REST JSON APIs
r = requests.post(url, data=payload, timeout=15)
```

Headers + API key

```
headers = {
    "Accept": "application/json",
    "X-Cisco-Meraki-API-Key": "fake-key-for-lab",
}
r = requests.get("https://httpbin.org/headers", headers=headers, timeout=15)
```

Basic auth

```
r = requests.get(
    "https://httpbin.org/basic-auth/admin/s3cret",
    auth=("admin", "s3cret"),
    timeout=15,
)
```

Bearer token

```
headers = {"Authorization": "Bearer TOKEN"}
r = requests.get("https://httpbin.org/bearer", headers=headers, timeout=15)
```

Branch on status (tie to 2.5)

```
if r.status_code == 200:
    data = r.json()
elif r.status_code == 401:
    raise SystemExit("Check auth")
else:
    raise SystemExit(f"Unexpected {r.status_code}: {r.text[:200]}")
```

Complete original script (pattern you should be able to write from docs)

```

"""Set a hostname via a fictional REST API – structure matches exam construction."""
import os
import requests

BASE = os.environ.get("API_BASE", "https://httpbin.org")
url = f"{BASE}/post"
headers = {"Accept": "application/json"}
body = {"hostname": "edge-01"}

response = requests.post(url, json=body, headers=headers, timeout=15)
print(response.status_code)
if response.status_code ≥ 400:
    print(response.text)
else:
    print(response.json())

```

Against httpbin this POST always echoes; against IOS XE RESTCONF you would change URL, `Accept: application/yang-data+json`, and `auth=`.

4. Exam-style understanding

You should be able to:

1. Write `requests.get` / `requests.post` with `timeout`.
2. Choose `params` vs `json` vs `headers` vs `auth` for a documented need.
3. Read `status_code` before trusting `.json()`.
4. Explain `json=` vs `data=`.
5. Assemble auth from 2.7 into the same script.

Original practice: Docs: GET `/organizations/{id}/devices`, header API key, query `perPage=10`.

Construct:

```

requests.get(
    f"https://api.meraki.com/api/v1/organizations/{org}/devices",
    headers={"X-Cisco-Meraki-API-Key": key},
    params={"perPage": 10},
    timeout=15,
)

```

Missing `params`, putting `perPage` in JSON on a GET, or using Basic instead of the key header are construction failures.

5. Hands-on exercise

```
cd "C:\Users\Reydel\Documents\04_Learning\CCNA Automation"
.\.venv\Scripts\Activate.ps1
cd labs\03_rest_api
python rest_client.py
```

Then **write from scratch** (do not copy-paste blindly) a file `my_get.py` in that folder:

1. GET <https://httpbin.org/get> with `params={"lab": "2.9"}`.
2. Print `status_code` and `json()["args"]`.
3. Add `timeout=15`.
4. Extend: POST a dict of one interface (from `labs/02_data_formats/interfaces.json` first object) to `/post` using `json=`.
5. Add a 401 case: GET `/basic-auth/admin/s3cret` without auth; print a message using the same logic as `troubleshoot_http.py`.

Read <https://requests.readthedocs.io/> for GET query strings and JSON POST. When this is fluent, Domain 3 Cisco platform APIs are the same script with different URLs and headers from <https://developer.cisco.com/>.

Domain 1 and 2 study close-out

You should now be able to move data from **XML/JSON/YAML** into Python, structure code and tests, talk process and Git with your hands on a repo, and drive **REST** with documented requests, HTTP literacy, auth, and `requests`.

Minimum lab pass for these two domains

Objective cluster	Command / action
1.1–1.2	<code>python labs/02_data_formats/parse_formats.py</code>
1.3, 1.5	<code>python -m unittest labs/01_python_basics/test_subnet.py</code> and run <code>functions_classes_modules.py</code>
1.7–1.8	Checklist in <code>labs/04_git/workflow.yaml</code> ; read <code>labs/04_git/example.diff</code>
2.1, 2.4, 2.6, 2.7, 2.9	<code>python labs/03_rest_api/rest_client.py</code>
2.5	<code>python labs/03_rest_api/troubleshoot_http.py</code>

Bookmark set

- <https://developer.cisco.com/>
- <https://httpbin.org>
- <https://docs.python.org/3/library/json.html>
- <https://git-scm.com/docs>
- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- <https://requests.readthedocs.io/>

Next domains reuse this chapter constantly: Cisco platform APIs are 2.1+2.7+2.9 with vendor paths; NETCONF is XML + RPC; Ansible is YAML; CI is Git + tests. If a later topic feels opaque, the gap is often here — especially Domain 2.